Fill in group number and member names:

In [28]:
```
GROUP = "11"
NAME1 = "Jakob Söderberg"
NAME2 = "Jonathan Möller"
```

Optimization for learning - FRTN50

Assignment 2

The purpose of this assignment is to empirically explore some convergence and implicit regularization properties of the stochastic gradient decent (SGD) method and some of its variants. These properties are mainly explored by considering simple polynomial fitting problems in $\mathbb{R}$ to facilitate easy comparison with known theoretical results and easy visualization. We will also compare the same fitting problem with multi-layer perceptrons also known as fully connected feedforward neural networks.

Below are the tasks that you need to solve. Keep this in mind:

- Carefully motivate every step in your calculations.
- Use figures and tables to motivate your answers.
- Figures must have appropriately labeled axes and must be referenced in the main text.
- Your code should be written in a quite general manner, i.e., if a question is slightly modified, it should only require slight modifications in your code as well.
- Comment your code well.
- Make sure you plot in such a way that small quantities are visible. In particular, use log-linear plots, where the quantity that should go to 0 is on the y-axis using logarithmic scale, and the iteration number k on the x-axis using linear scale.
- What you need to submit to Canvas:
  - This jupyter notebook containing your solutions.
  - An exported pdf version of the jupyter notebook.

Exercise 1: Polynomial fitting in $\mathbb{R}$ using least squares with a polynomial feature map

Consider fitting the points $\{(x_i, y_i)\}_{i=1}^N \subseteq \mathbb{R} \times \mathbb{R}$ to a polynomial model $m(\cdot;\theta) : \mathbb{R} \to \mathbb{R}$ using the method of least squares. Here, the model is given by

$$m(x;\theta) = \theta^T \phi(x)$$

where $\theta \in \mathbb{R}^{n+1}$ is the parameter of the model and $\phi : \mathbb{R} \to \mathbb{R}^{n+1}$ is a polynomial feature map given by

$$\phi(x) = \begin{bmatrix} 1 \\ x \\ \vdots \\ x^n \end{bmatrix}$$

for each $x \in \mathbb{R}$.

I.e. $m(\cdot;\theta)$ is a polynomial (in particular nonlinear) model of degree n and is linear in the parameter θ. If we compare to deep learning, the model $m(\cdot;\theta)$ would have a specific structure but would be nonlinear in the parameter θ in general.

The method of least squares involves solving the optimization problem

$$\underset{\theta\in\mathbb{R}^{n+1}}{\text{minimize}} \qquad \frac{1}{N}\sum_{i=1}^{N}(m(x_i;\theta)-y_i)^2 = \frac{1}{N}\sum_{i=1}^{N}\underbrace{(\theta^T\phi(x_i)-y_i)^2}_{=f_i(\theta)} = \frac{1}{N}\sum_{i=1}^{N} f_i(\theta)$$

giving an optimal point $\theta^* \in \mathbb{R}^{n+1}$, and letting $m(\cdot;\theta^*)$ be the fitted model. If we let

$$X = \begin{bmatrix} \phi(x_1)^T \\ \vdots \\ \phi(x_N)^T \end{bmatrix} \in \mathbb{R}^{N\times(n+1)}, \quad \text{and} \quad Y = \begin{bmatrix} y_1 \\ \vdots \\ y_N \end{bmatrix} \in \mathbb{R}^N,$$

optimization problem (1) can be written as

$$\underset{\theta\in\mathbb{R}^{n+1}}{\text{minimize}} \qquad \frac{1}{N}\|X\theta - Y\|_2^2 = f(\theta). \tag{1}$$

Since the model $m(\cdot;\theta)$ is linear in θ, the function f is convex in θ. This is not the case in deep learning, since the model $m(\cdot;\theta)$ would not be linear in θ in general.

If the points x_i in $\{(x_i,y_i)\}_{i=1}^N$ are distinct, which is the case in our analysis, it holds that

$$\text{rank } X = \min(N, n+1).$$

(See Vandermonde matrix for more info.) In particular,

1. if $N < n+1$, optimization problem (2) is underdetermined, XX^T is invertible and there exist $\theta^* \in \mathbb{R}^{n+1}$ such that $X\theta^* = Y$,
2. if $N > n+1$, optimization problem (2) is overdetermined and X^TX is invertible,
3. if $N = n+1$, both XX^T and X^TX are invertible and optimization problem (2) is neither underdetermined nor overdetermined.

You are allowed to assume these facts as given.

---

a) A model is sometimes said to be overparameterized if it has more parameters than data points. By the same token, a model is said to be underparameterized if it has

less parameters than data points.

- Suppose that our model is overparameterized. Which case of 1., 2. and 3. above does this corresponds to?
- Suppose that our model is underparameterized. Which case of 1., 2. and 3. above does this corresponds to?

Motivate your answer. It is crucial that you get this right to avoid confusion later.

Solution:

A model is overparameterized if it has too many parameters relative to the data. This means we have $N < n + 1$ meaning it is underdetermined, responding to case 1. The opposite holds for an underparameterized model, meaning we have $N > n + 1$ responding to case 2.

---

b) Find ∇f, ∇f_i and $\nabla^2 f$.

Carefully motivate your answer.

Solution:

Fill in your solution here!

$$\nabla f = \frac{2}{N} X^T (X\theta - Y)$$

$$\nabla f_i = 2(\theta^T \phi(x_i) - y_i)\phi(x_i)$$

$$\nabla^2 f = \frac{2}{N} X^T X$$

---

c) Since f is convex and differentiable, all optimal points $\theta^* \in \mathbb{R}^{n+1}$ to the least squares problem are fully characterized by the condition that

$$\nabla f(\theta^*) = 0. \tag{2}$$

Suppose that the least squares problem is underparameterized.

- Show that there exists a solution θ^*
- Conclude that it is unique
- Find a expression for $f(\theta^*)$

Carefully motivate your answer.

Solution:

We have by Fermat's rule that θ minimizes f if and only if:

$$0 \in \partial f(\theta)$$

Since f is differentaible we have that:

$$\partial f(\theta) = \{\nabla f(\theta)\}$$

From exercise 1a) we have:

$$\nabla f = X^T X\theta - X^T Y$$

Since the problem is underparameterized, we know that $X^T X$ is invertible, giving us the final expression:

$$\theta^* = (X^T X)^{-1} X^T Y$$

Suppose there existed another solution θ_1, then we would get:

$$X^T X\theta^* - X^T Y = X^T X\theta_1 - X^T Y$$

Which gives:

$$X^T X(\theta^* - \theta_1) = 0$$

Which holds if $\theta_* = \theta_1$ or somehow in theory if $\theta_* \neq \theta_1$ which we will show is not the case. Since $X^T X$ is invertible, we can rewrite it to:

$$(X^T X)^{-1} X^T X(\theta^* - \theta_1) = (X^T X)^{-1} 0$$

Which shows that $\theta_* = \theta_1$ is the only possible solution, thus making it unique.

$$f(\theta^*) = \frac{1}{N} \|X((X^T X)^{-1} X^T Y) - Y\|_2^2$$

---

d) Suppose instead that the least squares problem is overparameterized. Let $\theta^* \in \mathbb{R}^{n+1}$. Show that θ^* is a solution to the least squares problem if and only if $X\theta^* = Y$. I.e. show that:

- If θ^* is a solution to the least squares problem, then $X\theta^* = Y$.
- If $X\theta^* = Y$, then θ^* is a solution to the least squares problem.

Hence, the solution set is an affine set that can be written as $\Theta^* = \{\theta^* \in \mathbb{R}^{n+1} : X\theta^* = Y\}$.

- Prove that the set Θ^* is affine.

Carefully motivate your answer.

Hints: Note that $f(\theta) = \frac{1}{N} \|X\theta - Y\|_2^2 \geq 0$ for all $\theta \in \mathbb{R}^{n+1}$. Moreover, what can be said in the overparameterized case? What is the definition of an affine set?

Solution:

To prove it is a solution to the least squares problem we need to show that it's gradient is 0. The gradient was calculated in 1b) giving us the following:

$$X^T X\theta^* - X^T Y = 0$$

Rewriting gives us the following:

$$X^T(X\theta^* - Y) = 0$$

Since the problem is overparameterized we know that it is underdetermined and thus XX^T is invertible. By multiplying both sides with X, and multiplying both sides with $(XX^T)^1$ we get:

$$X\theta^* - Y = 0$$

Meaning

$$X\theta^* = Y$$

The definition of an affine set is: A set V is affine if for every $x, y \in V$ and $\alpha \in \mathbb{R}$

$$\alpha x + (1-\alpha)y \in V$$

Let $\theta_1, \theta_2 \in \Theta^*$ and $\alpha \in \mathbb{R}$. This means $X\theta_1 = Y$ and $X\theta_2 = Y$. If we pick an arbitrary point on the line passing through θ_1 and θ_2:

$$\alpha\theta_1 + (1-\alpha)\theta_2$$

Multiplying this with X, we get:

$$\alpha X\theta_1 + (1-\alpha)X\theta_2$$

Which is equal to:

$$\alpha Y + (1-\alpha)Y = Y$$

Which we know is in the set Θ^*, thus concluding the proof.

---

e) In the overparameterized case, the null space $\mathcal{N}(X) = \{\theta \in \mathbb{R}^{n+1} : X\theta = 0\}$ has dimension $\dim \mathcal{N}(X) = n + 1 - N > 0$. This can be shown using, e.g., the rank–nullity theorem. Suppose that $\hat{\theta} \in \mathbb{R}^{n+1}$ is some point that satisfies $X\hat{\theta} = Y$ and that $v \in \mathcal{N}(X)$.

- Show that $\hat{\theta} + v$ is a solution to the least squares problem.
- Show that $\hat{\theta} + \alpha v$ is a solution to the least squares problem for all $\alpha \in \mathbb{R}$. (This follows from the above but is included to emphasize the fact that, assuming $v \neq 0$, we can add an arbitrary large vector to $\hat{\theta}$ and it still remains a solution to the the least squares problem.)

As a consequence, note that an another characterization of the set of solutions of the least squares problem is given by $\Theta^* = \{\hat{\theta} + v : v \in \mathcal{N}(X)\}$. In particular, Θ^* has dimension $\dim \Theta^* = n + 1 - N > 0$. Therefore, the more overparameterized (larger $n + 1 - N$) the problem is, the large the solution set Θ^* becomes.

- Show that Θ^* and $\{\hat{\theta} + v : v \in \mathcal{N}(X)\}$ are equal. One way to show this is to show that all elements of Θ^* are in $\{\hat{\theta} + v : v \in \mathcal{N}(X)\}$, i.e. $\Theta^* \subseteq \{\hat{\theta} + v : v \in \mathcal{N}(X)\}$, and that all elements in $\{\hat{\theta} + v : v \in \mathcal{N}(X)\}$ are in Θ^*, i.e. $\{\hat{\theta} + v : v \in \mathcal{N}(X)\} \subseteq \Theta^*$.

Carefully motivate your answer.

Solution:

In the previous task we showed that if θ^* is a solution to the least squares problem, then $X\theta^* = Y$. If $\hat{\theta} + v$ is a solution to the problem, then $X(\hat{\theta} + v) = Y$ must hold. In order for it to be a solution to the least squares problem we need the gradient to be 0. The gradient is calculated in task 1b) as the following:

$$X^T X(\hat{\theta} + v) - X^T Y = 0$$

Rewriting gives us the following:

$$X^T(X(\hat{\theta} + v) - Y) = 0$$

Since the problem is overparameterized we know that it is underdetermined and thus XX^T is invertible. By multiplying both sides with X, and multiplying both sides with $(XX^T)^1$ we get:

$$X(\hat{\theta} + v) - Y = 0$$

Meaning

$$X(\hat{\theta} + v) = Y$$

Thus proving $(\hat{\theta} + v)$ is a solution to the problem. We can clearly see that multiplying a scalar α to v, the proof still holds.

To prove that Θ^* and $\{\hat{\theta} + v : v \in \mathcal{N}(X)\}$ are equal we can pick any fixed point $\Theta_0^* \in \Theta^*$ and it is trivially true that for any point in Θ^* it can be written as Θ_0^* plus a vector C. We know that:

$$X(\Theta^* + C) = Y$$

Since $X\Theta^* = Y$ it follows that $XC = 0$ thus $C \in \mathcal{N}(X)$. To complete the proof we will show the other direction.

Given that $v \in \mathcal{N}(X)$ and that $X\hat{\theta} = Y$, we know that $X(\hat{\theta} + v) = Y$, which concludes the proof.

---

f) Suppose that the least squares problem is overparameterized.

Why can we not use the solution from **c)** in this case?

Solution:

Because we don't know if X^TX is invertible.

---

g) Suppose that the least squares problem is overparameterized.

We know from **d)** and **e)** that the set of all solutions Θ^* is an affine set with infinitely many elements. One way to pick a solution is to start with some candidate point $a \in \mathbb{R}^{n+1}$ and project it onto Θ^*. Since Θ^* is closed, convex and nonempty, the projection point exists and is unique, and can be found by solving the optimization problem

$$\begin{array}{ll} \underset{\theta \in \mathbb{R}^{n+1}}{\text{minimize}} & \dfrac{1}{2}\|\theta - a\|_2^2 \\ \text{subject to} & \theta \in \Theta^*. \end{array} \tag{3}$$

We call this the Euclidean projection problem. Another way to write the Euclidean projection problem is

$$\underset{\theta \in \mathbb{R}^{n+1}}{\text{minimize}} \quad \frac{1}{2}\|\theta - a\|_2^2 + \iota_{\Theta^*}(\theta) \tag{4}$$

where ι_{Θ^*} is the indicator function of the set Θ^*. By Fermat's rule, $\theta^* \in \mathbb{R}^{n+1}$ is an optimal point to the Euclidean projection problem if and only if the optimality condition

$$0 \in \theta^* - a + \partial \iota_{\Theta^*}(\theta^*)$$

holds, by convexity and since CQ holds. Note that in the exercise compendium, we show that the subdifferential of the indicator function of a set is the normal cone to that set. In particular, in our case we have that

$$\begin{aligned} \partial \iota_{\Theta^*}(\theta^*) &= N_{\Theta^*}(\theta^*) \\ &= \begin{cases} \left\{ s \in \mathbb{R}^n : \forall \eta \in \Theta^*,\, s^T(\eta - \theta^*) \leq 0 \right\} & \text{if } \theta^* \in \Theta^* \\ \emptyset & \text{if } \theta^* \notin \Theta^* \end{cases} \\ &= \begin{cases} \left\{ s \in \mathbb{R}^n : \forall v \in \mathcal{N}(X),\, s^T v = 0 \right\} & \text{if } \theta^* \in \Theta^* \\ \emptyset & \text{if } \theta^* \notin \Theta^* \end{cases} \\ &= \begin{cases} \mathcal{R}(X^T) & \text{if } \theta^* \in \Theta^* \\ \emptyset & \text{if } \theta^* \notin \Theta^* \end{cases} \\ &= \begin{cases} \left\{ X^T\lambda : \lambda \in \mathbb{R}^N \right\} & \text{if } \theta^* \in \Theta^* \\ \emptyset & \text{if } \theta^* \notin \Theta^*. \end{cases} \end{aligned}$$

If we combine this with the optimality condition above we get that θ^* is an optimal point to the Euclidean projection problem if and only if there exists a $\lambda^* \in \mathbb{R}^N$ such that

$$0 = \theta^* - a + X^T\lambda^* \quad \text{and} \quad X\theta^* = Y. \tag{5}$$

Use this last optimality condition to show that $\theta^* = a - X^T(XX^T)^{-1}(Xa - Y)$ is the optimal point for the Euclidean projection problem. We also call this θ^* the

Euclidean projection (of a onto Θ^*).

Carefully motivate your answer.

Remark: We have a generalization: Suppose that $H \in \mathbb{S}_{++}^{n+1}$ and let $\|\theta\|_H^2 = \theta^T H \theta$. Then the scaled projection problem

$$\begin{array}{ll} \underset{\theta \in \mathbb{R}^{n+1}}{\text{minimize}} & \frac{1}{2}\|\theta - a\|_H^2 \\ \text{subject to} & \theta \in \Theta^*. \end{array} \tag{6}$$

has the solution $\theta^* = a - H^{-1}X^T(XH^{-1}X^T)^{-1}(Xa - Y)$. We call this θ^* the scaled projection (of a onto Θ^*). You are not allowed to used this generalization to solve this problem. However, it will be used later.

Solution:

$$0 = \theta^* - a + X^T\lambda^* \tag{7}$$
$$X(a - \theta^*) = XX^T\lambda^* \tag{8}$$

Since the problem is overparameterized we know that $(XX^T)^{-1}$ exists, we can get the following:

$$\lambda^* = (XX^T)^{-1}X(a - \theta^*) \tag{9}$$

Plugging this in to the expression gives:

$$0 = \theta^* - a + X^T(XX^T)^{-1}X(a - \theta^*) \tag{10}$$

Which by rewriting and the fact that $X\theta^* = Y$ gives the final expression:

$$\theta^* = a - X^T(XX^T)^{-1}(Xa - Y) \tag{11}$$

---

h) Show that $f : \mathbb{R}^{n+1} \to \mathbb{R}$ is β-smooth, i.e., ∇f is β-Lipschitz continuous, with constant

$$\beta = \frac{2}{N}\left\|X^TX\right\|_2. \tag{12}$$

Recall that ∇f is said to be β-Lipschitz continuous if

$$\|\nabla f(\theta_1) - \nabla f(\theta_2)\|_2 \leq \beta\|\theta_1 - \theta_2\|_2, \quad \forall \theta_1, \theta_2 \in \mathbb{R}^{n+1}. \tag{13}$$

Hint: Recall that the spectral norm of a matrix $A \in \mathbb{R}^{l \times m}$ is defined as

$$\|A\|_2 = \sup_{z \in \mathbb{R}^m \setminus \{0\}} \frac{\|Az\|_2}{\|z\|_2}. \tag{14}$$

In particular,

$$\|Az\|_2 \leq \|A\|_2 \|z\|_2, \quad \forall z \in \mathbb{R}^m. \tag{15}$$

Remark: Note that $\|\cdot\|_2$ is the **spectral norm** and not the **Frobenius norm** - this difference is very important to consider in your implementation below where you will use this β. Moreover, note that β depends on X, and will vary as X varies.

Solution:

For two points θ_1, θ_2 we have:

$$\|\nabla f(\theta_1) - \nabla f(\theta_2)\|_2 = \left\| \frac{2}{N} X^T (X\theta_1 - Y) - (\frac{2}{N} X^T (X\theta_2 - Y)) \right\|_2 \tag{16}$$

$$= \left\| \frac{2}{N} X^T X (\theta_1 - \theta_2) \right\|_2 \tag{17}$$

$$\leq \left\| \frac{2}{N} X^T X \right\|_2 \|\theta_1 - \theta_2\|_2 \tag{18}$$

$$= \frac{2}{N} \left\| X^T X \right\|_2 \|\theta_1 - \theta_2\|_2 \tag{19}$$

Which shows that ∇f is β-Lipschitz continuous, with constant $\beta = \frac{2}{N} \left\| X^T X \right\|_2$

Exercise 2: Implementing GD and SGD for polynomial fitting

Remark: You should solve this exercise using `numpy` primarily.

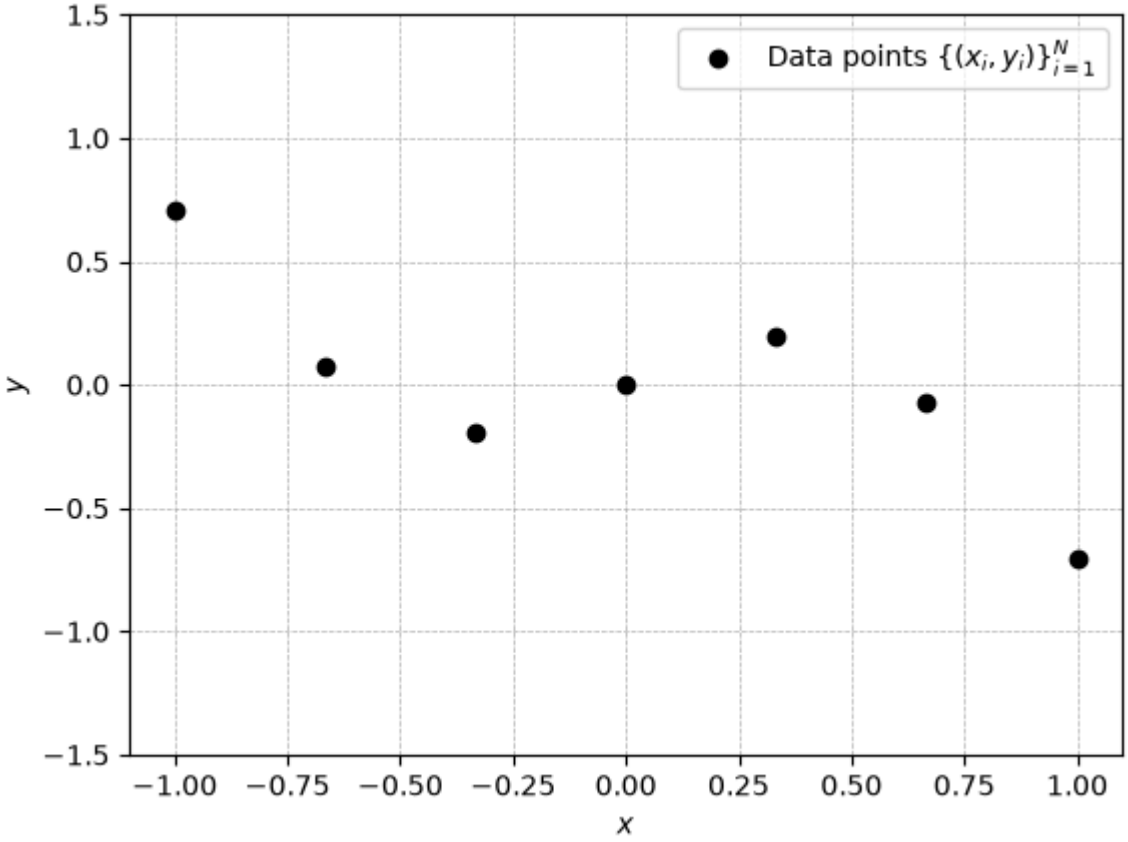
Consider the data point $\{(x_i, y_i)\}_{i=1}^N \subseteq \mathbb{R} \times \mathbb{R}$, where $N = 7$, below:

In [29]:
```
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

N = 7 # the number of data points
xs = np.linspace(-1, 1, num=N) # the x_i's
ys = np.sin(5*xs)*np.exp(np.abs(2*xs)) / 10 # the y_i's

fig, axes = plt.subplots(dpi=100)
axes.plot(xs,ys,'ko')
axes.legend([r'Data points $\{(x_{i},y_{i})\}_{i=1}^{N}$'])
axes.set_xlabel('$x$')
axes.set_ylabel('$y$')
axes.grid(linestyle='dashed', linewidth=0.5)
axes.set_ylim(-1.5, 1.5)
```

Out[29]:
```
(-1.5, 1.5)
```



We wish to fit a polynomial model $m(\cdot;\theta)$ of degree $n = 3$ (overdetermined) and $n = 9$ (underdetermined) to the data points $\{(x_i, y_i)\}_{i=1}^N$ by solving optimization problem (1) or equivalently (2) in **Exercise 1** above.

---

a) First, do this by implementing the GD method. The GD method is given by

- Pick $\theta_0 \in \mathbb{R}^{n+1}$
- For $k \geq 0$:
  - Set $\theta_{k+1} = \theta_k - \gamma_k \nabla f(\theta_k)$ for some $\gamma_k > 0$.

Pick $\theta_0 = 0$ as the initial point, and try the constant step size

1. $\gamma_k = 1/\beta$,

and run for 10^5 iterations. Here, β is the smoothness parameter from **Exercise 1** above. Plot, in log-linear scale, both

- $f(\theta_k) - f(\theta^*)$ and
- $f(\bar{\theta}_k) - f(\theta^*)$ where $\bar{\theta}_k = \frac{1}{k+1}\sum_{l=0}^{k} \theta_l$, (Hint: use sequential average computation)

as a function of iteration number k. Give comments:

- Describe any difference between the $n = 3$ (overdetermined) and $n = 9$ (underdetermined) cases.

Hint: What is $f(\theta^*)$ in the overdetermined and underdetermined case, respectively?

Solution:

In [30]:
```
def f(theta, X, Y):
  return np.linalg.norm(X @ theta - Y)**2 / len(Y)

def grad_f(theta, X, Y):
  return 2 / len(Y) * X.T @ (X @ theta - Y)

def gradient_descent(theta, n, iters =10**5):
  X = np.vander(xs, n+1, increasing=True)
  Y = ys
  beta = 2 / N * np.linalg.eigvalsh(X.T @ X).max()

  gamma = 1 / beta

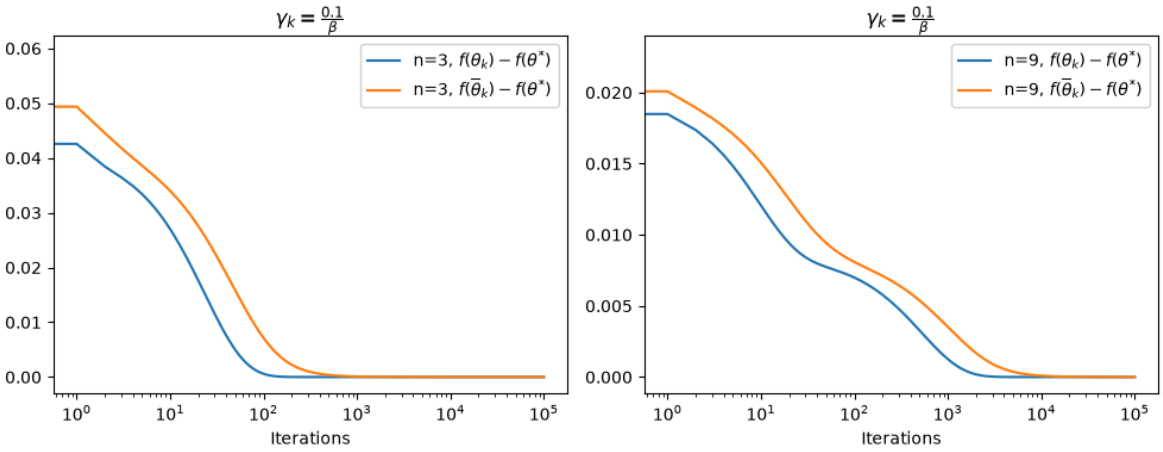
  if N < n + 1:
    a = np.zeros(n+1)
    theta_star = a - X.T @ np.linalg.inv(X @ X.T) @ (X @ a - Y)
  else:
    theta_star = np.linalg.inv(X.T @ X) @ X.T @ Y
  theta = np.zeros(n+1)
  theta_k = np.zeros(n+1)
  res_list = []
  res_hat_list = []

  for i in range(iters):
    theta_next = theta - gamma * grad_f(theta, X, Y)
    theta = theta_next
    theta_k += (theta_next - theta_k) / (i + 1)
    res = f(theta_next, X, Y) - f(theta_star, X, Y)
    res_list.append(res)
    res_hat = f(theta_k, X, Y) - f(theta_star, X, Y)
    res_hat_list.append(res_hat)

  return res_list, res_hat_list

parameters = [3, 9]
fig, axs = plt.subplots(1, 2, figsize=(10, 4))

for j, n in enumerate(parameters):
  res_list, res_hat_list = gradient_descent(np.zeros(n + 1), n)
  axs[j].semilogx(res_list, label=f"n={n}, {r"$f(\theta_{k})-f(\theta^{*}
  axs[j].plot(res_hat_list, label=f"n={n}, {r"$f(\overline{\theta}_{k})-f
  axs[j].set_xlabel("Iterations")
  axs[j].legend()
  axs[j].set_title(r"$\gamma_k = \frac{0.1}{\beta}$")
plt.tight_layout()
plt.show()
```



---

b) Next, do this by implementing the SGD method. The SGD method is given by

- Pick $\theta_0 \in \mathbb{R}^{n+1}$
- For $k \geq 0$:
  - Pick $i \in \{1, \ldots, N\}$ uniformly at random
  - Set $\theta_{k+1} = \theta_k - \gamma_k \nabla f_i(\theta_k)$ for some $\gamma_k > 0$.

Pick $\theta_0 = 0$ and try the step sizes

1. $\gamma_k = \frac{0.1}{\beta}$,
2. $\gamma_k = \frac{0.1}{\beta(k+1)}$,
3. $\gamma_k = \frac{0.1}{\beta(\lfloor k/1000 \rfloor + 1)}$, where $\lfloor \cdot \rfloor$ is the floor function,

and run for $10^5 N$ iterations. Plot, in log-linear scale, both

- $f(\theta_k) - f(\theta^*)$ and
- $f(\bar{\theta}_k) - f(\theta^*)$ where $\bar{\theta}_k = \frac{1}{k+1} \sum_{l=0}^{k} \theta_l$,

as a function of iteration number k. Give comments:

- Does the step sizes matter?
- Is there a difference between the $n = 3$ (overdetermined) and $n = 9$ (underdetermined) cases.
- Which combination worked best?
- Which combination of γ_k and n (i.e. overdetermined or underdetermined) corresponds the most to training of neural networks? This question qualitative and has no unique answer.

Solution:

The step size clearly matters from the plot. $\gamma_k = \frac{0.1}{\beta(k+1)}$ does not manage to converge using 2000 iterations while the other two step sizes converges. The plots show that there is a minimal difference between the overdetermined and underdetermined case. The plots show that $n = 3$ with $\gamma_k = \frac{0.1}{\beta(\lfloor k/1000 \rfloor + 1)}$ converges the fastest, thus making it the best combination. Training neural networks

is mostly done on underdetermined models. For the underdetermined models, the step-size $\gamma_k = \frac{0.1}{\beta}$ appeared to converge fastest, making it the best option for our case.

In [31]:
```
def f(theta, X, Y):
  return np.linalg.norm(X @ theta - Y)**2 / len(Y)

def grad_f(theta, X, Y):
  return 2 / len(Y) * X.T @ (X @ theta - Y)

def partial_grad_f(theta, X, Y, i):
  return 2 * (X[i] @ theta - Y[i]) * X[i]

def stochastic_gradient_descent(theta, schedule, n, iters =10**5):
  X = np.vander(xs, n+1, increasing=True)
  Y = ys
  beta = 2 / N * np.linalg.eigvalsh(X.T @ X).max()

  if N < n + 1:
    a = np.zeros(n+1)
    theta_star = a - X.T @ np.linalg.inv(X @ X.T) @ (X @ a - Y)
  else:
    theta_star = np.linalg.inv(X.T @ X) @ X.T @ Y

  theta_k = np.zeros(n+1)
  f_star = f(theta_star, X, Y)
  res_list = []
  res_hat_list = []

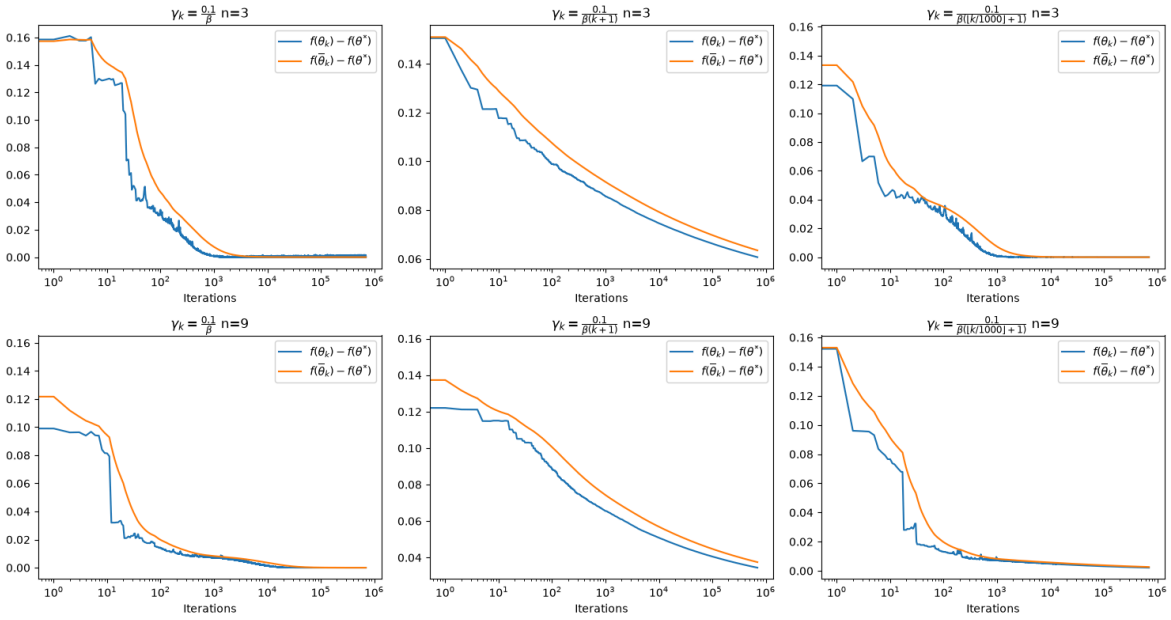
  for i in range(iters*len(Y)):
    if schedule == "floor":
      gamma = 0.1 / (beta * (i // 1000 + 1))
    elif schedule == "linear_decay":
      gamma = 0.1 / (beta * (i + 1))
    else:
      gamma = 0.1 / beta
    j = np.random.randint(0, len(Y))
    theta_next = theta - gamma * partial_grad_f(theta, X, Y, j)
    theta = theta_next
    theta_k += (theta_next - theta_k) / (i + 1)

    res = f(theta_next, X, Y) - f_star
    res_list.append(res)
    res_hat = f(theta_k, X, Y) - f_star
    res_hat_list.append(res_hat)

  return res_list, res_hat_list

gammas = {"constant": r"$\gamma_k = \frac{0.1}{\beta}$", "linear_decay":
parameters = [3, 9]
fig, axs = plt.subplots(2, 3, figsize=(15, 8))
for i, (schedule, title) in enumerate(gammas.items()):
  for j, n in enumerate(parameters):
    res_list, res_hat_list = stochastic_gradient_descent(np.zeros(n + 1),
    axs[j, i].semilogx(res_list, label=f"{r"$f(\theta_{k})-f(\theta^{*})$
    axs[j, i].semilogx(res_hat_list, label=f"{r"$f(\overline{\theta}_{k})
    axs[j, i].set_xlabel("Iterations")
    axs[j, i].legend()
```

```
    axs[j, i].set_title(f"{title} n={n}")
plt.tight_layout()
plt.show()
```

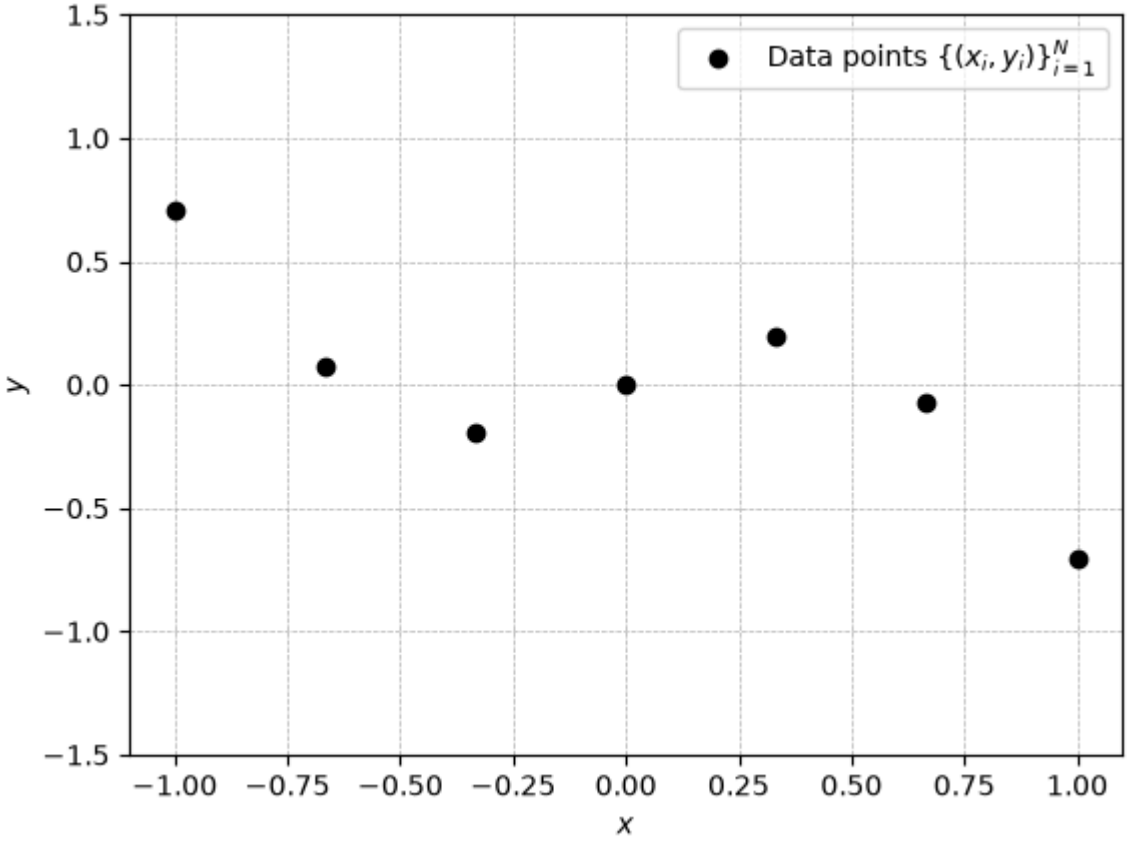


Exercise 3: Fitting points in $\mathbb{R}$ using fully connected feedforward neural networks

Again, we consider the data points $\{(x_i, y_i)\}_{i=1}^N \subseteq \mathbb{R} \times \mathbb{R}$, where $N = 7$, below:

In [32]:
```
N = 7 # the number of data points
xs = np.linspace(-1, 1, num=N) # the x_i's
ys = np.sin(5*xs)*np.exp(np.abs(2*xs)) / 10 # the y_i's

fig, axes = plt.subplots(dpi=100)
axes.plot(xs,ys,'ko')
axes.legend([r'Data points $\{(x_{i},y_{i})\}_{i=1}^{N}$'])
axes.set_xlabel('$x$')
axes.set_ylabel('$y$')
axes.grid(linestyle='dashed', linewidth=0.5)
axes.set_ylim([-1.5, 1.5]);
```

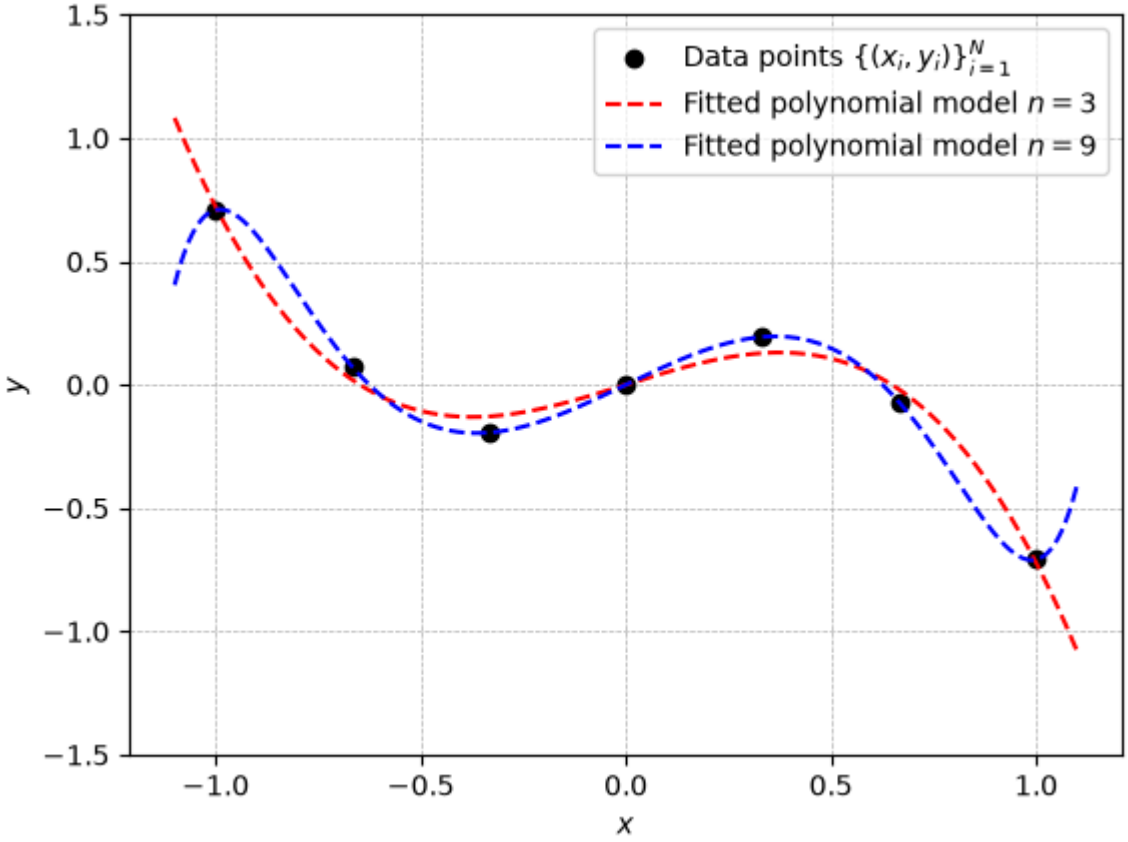


Polynomial fitting as in **Exercise 1** and **Exercise 2** gives:

In [33]:
```
theta_star_n_9 = np.array([0,  0.8, 0, -1.9, 0, -0.583957507, 0, 0.28, 0,
theta_star_n_3 = np.array([0,  0.523557283,  0, -1.24520755])
x = np.linspace(-1.1, 1.1, num = 1000)

fig, axes = plt.subplots(dpi=100)
axes.plot(xs,ys,'ko')
axes.plot(x, np.polyval(np.flip(theta_star_n_3), x),'r--')
axes.plot(x, np.polyval(np.flip(theta_star_n_9), x),'b--')
axes.legend([r'Data points $\{(x_{i},y_{i})\}_{i=1}^{N}$', r'Fitted polyn
axes.set_xlabel('$x$')
axes.set_ylabel('$y$')
axes.grid(linestyle='dashed', linewidth=0.5)
axes.set_ylim(-1.5, 1.5)
```

Out[33]: (-1.5, 1.5)



In this exercise, we are going to explore fitting of these data points using fully connected feedforward neural networks. We will do this by considering the problem

$$\underset{\theta}{\text{minimize}} \frac{1}{N} \sum_{i=1}^{N} L(m(x_i; \theta), y_i) \tag{3}$$

where $L : \mathbb{R} \times \mathbb{R} \to \mathbb{R}$ is the least squares loss, i.e.

$$L(u, y) = (u - y)^2$$

for each $u, y \in \mathbb{R}$ and $m(\cdot; \theta) : \mathbb{R} \to \mathbb{R}$ is a fully connected feedforward neural network model, i.e.

$$m(x; \theta) = W_D \sigma_{D-1}(W_{D-1} \sigma_{D-2}(\dots \sigma_1(W_1 x + b_1) \dots) + b_{D-1}) + b_D$$

for each $x \in \mathbb{R}$, where σ_i is an activation functions that act elements-wise, for each $i = 1, \dots, D-1$, and

$$\theta = (W_1, b_1, \dots, W_D, b_D)$$

such that

$$\begin{cases} W_1 \in \mathbb{R}^{f_1 \times 1} & \text{and} \quad b_1 \in \mathbb{R}^{f_1} \\ W_i \in \mathbb{R}^{f_i \times f_{i-1}} & \text{and} \quad b_i \in \mathbb{R}^{f_i} \quad \text{for} \quad i = 2, \dots, D-1 \\ W_D \in \mathbb{R}^{1 \times f_{D-1}} & \text{and} \quad b_D \in \mathbb{R}. \end{cases}$$

In particular, we will study how the trained model $m(\cdot; \theta)$ depends on:

- The widths (f_i) and depth (D)

- Different activation functions (σ_i)
- Different initial points θ_0 used in training
- Different training algorithms such as SGD and and Adam

We will also study:

- How the loss landscape of (3) looks along different directions at an "optimal point" of (3). (Note that we in general have no guarantees that our optimization algorithms will reach a true optimal point of (3) since the problem in nonconvex in general.)

Remark: We will train the models using `pytorch`. `pytorch` is a widely used library or *module* for doing machine learning in python. It allows the user to construct, train and evaluate neural networks.

- You will need to install `pytorch`

Take a quick look at `pytorch`'s

- tutorials
- documentation

`pytorch` expects to get data from a `DataLoader` object. Below we create a `DataLoader` and provide it with our data:

In [34]:
```
import torch
from torch import nn
import torch.nn.functional as F
from torch.utils.data import TensorDataset
from torch.utils.data import DataLoader
from torch import optim

torch_xs = torch.tensor(xs.reshape((N,1)), dtype=torch.float32)
torch_ys = torch.tensor(ys.reshape((N,1)), dtype=torch.float32)
dataset = TensorDataset(torch_xs, torch_ys)
data_loader = DataLoader(dataset, batch_size=len(xs)) # other batch sizes
```

Typically in `pytorch`, models are defined as a child of the `nn.Module` class. Below, we create a model with:

- Depth $D = 2$
- Width $f_1 = 10$
- ReLU as activation function

In [35]:
```
class model(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear1 = nn.Linear(in_features=1, out_features=10)
        self.linear2 = nn.Linear(in_features=10, out_features=1)

    def forward(self, x):
        x = self.linear1(x)
```

```
        x = F.relu(x)
        x = self.linear2(x)
        return x

# Create an instance of the model class.
model = model()
```

In machine learning, the objective function in (3) is sometimes itself called a loss function. In fact, it is already implemented in `pytorch` and we create the corresponding loss function object below:

In [36]:
```
loss_fn = nn.MSELoss()
```

`torch.optim` is a `pytorch` package implementing various optimization algorithms. Below we create an optimizer object that allows us to take a SGD step with step-size/learning rate equal to 0.05.

In [37]:
```
optimizer = optim.SGD(model.parameters(), lr=0.05)
```

Once we have defined the data loader, the model, the loss function and the optimizer as above, we can train the network by simply following these steps in a loop:

1. Sample a batch of data from our dataset
2. Compute the model's prediction on the batch
3. Compute the loss of the prediction w.r.t. ground-truth
4. Backpropagate the loss through the model's parameters
5. Perform one training step.

We will do this for 2000 epochs.

In [38]:
```
for epoch in range(2000):
    losses_in_epoch = []
    for batch in data_loader:

        # 1. These are the sampled batches of inputs and ground-truth
        batch_x, batch_y = batch

        # 2. Compute the model's prediction on the batch
        pred = model(batch_x)

        # 3. Compute the loss of the prediction w.r.t. ground-truth
        loss = loss_fn(pred, batch_y)

        # Save losses in a list for averaging later (not sctrictly necess
        losses_in_epoch.append(loss)

        # 4. Backpropagation
        loss.backward()

        # 5. One step of SGD
        optimizer.step()

        # Zero the gradients computed in the backpropagation, for startin
        optimizer.zero_grad()
```

```
    # Every 100th epoch, display progress
    if epoch % 100 == 0:
        print('Epoch: {}\tLoss: {}'.format(epoch, sum(losses_in_epoch)/le
```

```
Epoch: 0        Loss: 0.08799899369478226
Epoch: 100      Loss: 0.06435150653123856
Epoch: 200      Loss: 0.05912712588906288
Epoch: 300      Loss: 0.056786078959703445
Epoch: 400      Loss: 0.0540691576898098
Epoch: 500      Loss: 0.05089534446597099
Epoch: 600      Loss: 0.047287728637456894
Epoch: 700      Loss: 0.04325321689248085
Epoch: 800      Loss: 0.03891043737530708
Epoch: 900      Loss: 0.03438744693994522
Epoch: 1000     Loss: 0.02985096536576748
Epoch: 1100     Loss: 0.025478094816207886
Epoch: 1200     Loss: 0.021467629820108414
Epoch: 1300     Loss: 0.017890581861138344
Epoch: 1400     Loss: 0.014788552187383175
Epoch: 1500     Loss: 0.01219168957322836
Epoch: 1600     Loss: 0.010061723180115223
Epoch: 1700     Loss: 0.00830917339771986
Epoch: 1800     Loss: 0.0068475934676826
Epoch: 1900     Loss: 0.005627898965030909
```

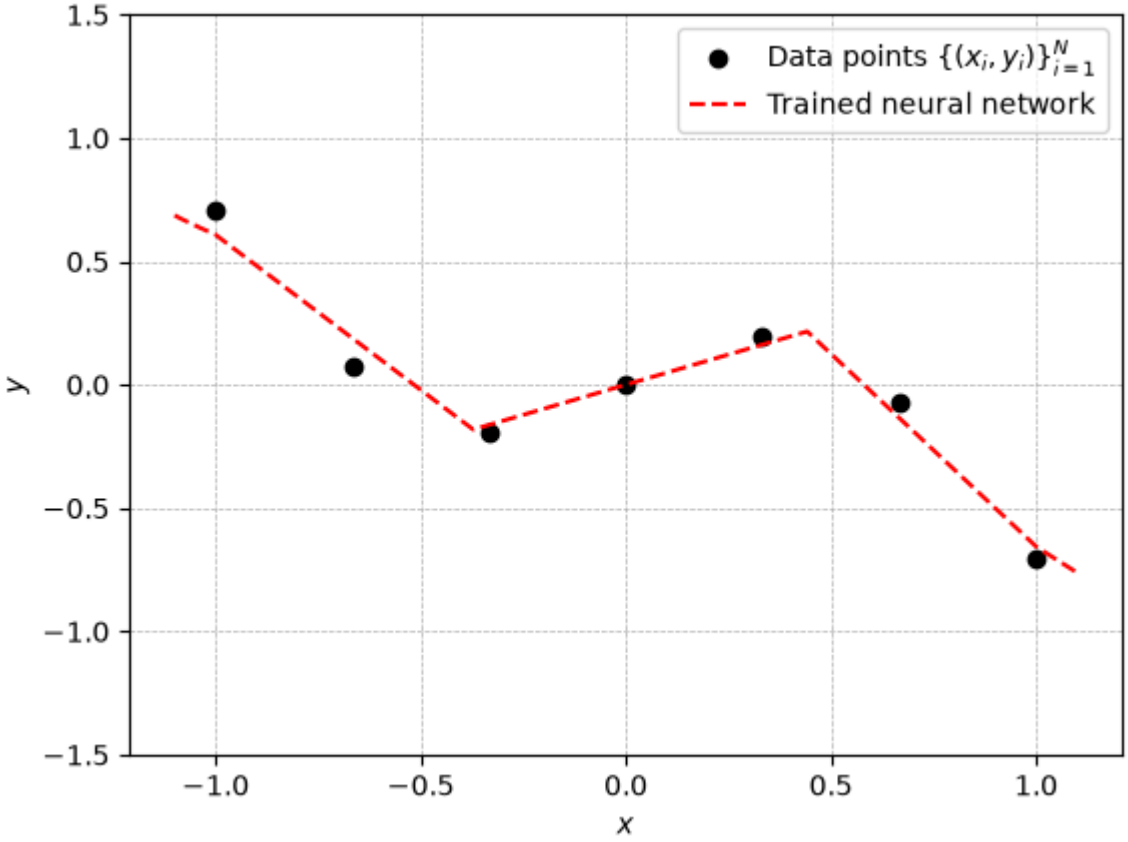
Below we plot trained model:

In [39]:
```
with torch.no_grad():
    x = np.linspace(-1.1, 1.1, num = 1000)
    torch_x = torch.tensor(x.reshape((1000,1)), dtype=torch.float32)
    y_pred = model(torch_x).numpy()

    fig, axes = plt.subplots(dpi=100)
    axes.plot(xs,ys,'ko')
    axes.plot(x, y_pred,'r--')
    axes.legend([r'Data points $\{(x_{i},y_{i})\}_{i=1}^{N}$', r'Trained
    axes.set_xlabel('$x$')
    axes.set_ylabel('$y$')
    axes.grid(linestyle='dashed', linewidth=0.5)
    axes.set_ylim([-1.5, 1.5])

    print("Traing loss: {:.5f}".format(loss_fn(model(torch_xs),torch_ys).
```

```
Traing loss: 0.00462
```



a) First, we experiment with the widths (f_i) and depth (D).

Choose widths (f_i) and depth (D) such that:

- the network becomes underparameterized (motive why it is underparameterized)
- the network becomes overparameterized (motive why it is overparameterized)

and train the model as above.

In this subproblem, keep the activation functions fixed as ReLU. However, you are free to experiment with the learning rate and the number of epoch.

How does these two cases differ? Give a qualitative and quantitative analysis.

Solution:

Creating a model with $D = 2$ and $f_1 = f_2 = 2$ gives us a model with 4 parameters and thus underparameterized. When training the model, we see that by the use of least squares it finds an optimal solution which is constant through the rest of the training epochs.

In [40]:
```
class UnderparameterizedModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear1 = nn.Linear(in_features=1, out_features=2)
        self.linear2 = nn.Linear(in_features=2, out_features=1)

    def forward(self, x):
        x = self.linear1(x)
        x = F.relu(x)
```

```
        x = self.linear2(x)
        return x


class OverparameterizedModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear1 = nn.Linear(in_features=1, out_features=10)
        self.linear2 = nn.Linear(in_features=10, out_features=20)
        self.linear3 = nn.Linear(in_features=20, out_features=10)
        self.linear4 = nn.Linear(in_features=10, out_features=1)

    def forward(self, x):
        x = self.linear1(x)
        x = F.relu(x)
        x = self.linear2(x)
        x = F.relu(x)
        x = self.linear3(x)
        x = F.relu(x)
        x = self.linear4(x)

        return x

def train(model, iters=2000, log=True):
    loss_fn = nn.MSELoss()
    optimizer = optim.SGD(model.parameters(), lr=0.05)
    loss_res = []
    for epoch in range(iters):
        losses_in_epoch = []
        for batch in data_loader:

            # 1. These are the sampled batches of inputs and ground-truth
            batch_x, batch_y = batch

            # 2. Compute the model's prediction on the batch
            pred = model(batch_x)

            # 3. Compute the loss of the prediction w.r.t. ground-truth
            loss = loss_fn(pred, batch_y)

            # Save losses in a list for averaging later (not sctrictly ne
            losses_in_epoch.append(loss.item())

            # 4. Backpropagation
            loss.backward()

            # 5. One step of SGD
            optimizer.step()

            # Zero the gradients computed in the backpropagation, for sta
            optimizer.zero_grad()

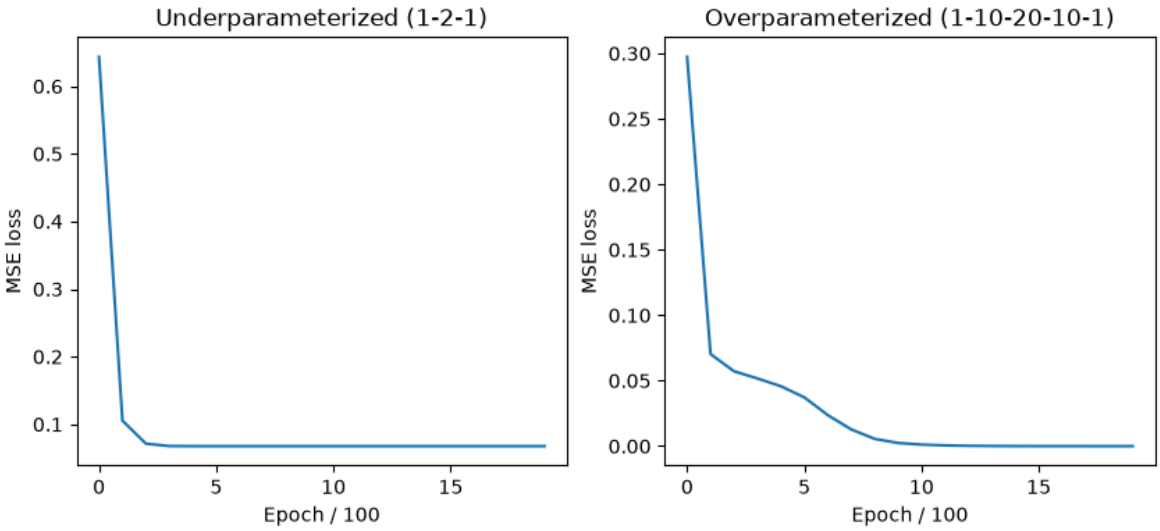
        # Every 100th epoch, display progress
        if epoch % 100 == 0:
            loss_res.append(sum(losses_in_epoch)/len(losses_in_epoch))
            if log:
                print('Epoch: {}\tLoss: {}'.format(epoch, sum(losses_in_e
    return loss_res

underparameterized_model = UnderparameterizedModel()
```

```
overparameterized_model = OverparameterizedModel()

underparameterized_loss_res = train(underparameterized_model, log=False)
overparameterized_loss_res = train(overparameterized_model, log=False)
fig, axs = plt.subplots(1, 2, figsize=(10, 4))
axs[0].plot(underparameterized_loss_res)
axs[0].set_title("Underparameterized (1-2-1)")
axs[1].plot(overparameterized_loss_res)
axs[1].set_title("Overparameterized (1-10-20-10-1)")
for ax in axs:
    ax.set_xlabel("Epoch / 100")
    ax.set_ylabel("MSE loss")

plt.show()
```



b) Second, we will try out different activation functions (σ_i).

Train two overparameterized networks with the same widths and depth, using

- ReLU activation at each layer for one of the networks, and
- 4 * sigmod activation at each layer for the other network.

You are free to experiment with the learning rate and the number of epoch.

How does these two cases differ? Give a qualitative and quantitative analysis.

Solution:

Using sigmoid we get a smooth function with no kinks while when using ReLU we get a piecewise linear function due to the characteristics of the activation functions.

In [41]:
```
class SigmoidModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear1 = nn.Linear(in_features=1, out_features=10)
        self.linear2 = nn.Linear(in_features=10, out_features=20)
        self.linear3 = nn.Linear(in_features=20, out_features=10)
        self.linear4 = nn.Linear(in_features=10, out_features=1)

    def forward(self, x):
        x = self.linear1(x)
```

```
        x = 4 * F.sigmoid(x)
        x = self.linear2(x)
        x = 4 * F.sigmoid(x)
        x = self.linear3(x)
        x = 4 * F.sigmoid(x)
        x = self.linear4(x)
        return x


class ReluModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear1 = nn.Linear(in_features=1, out_features=10)
        self.linear2 = nn.Linear(in_features=10, out_features=20)
        self.linear3 = nn.Linear(in_features=20, out_features=10)
        self.linear4 = nn.Linear(in_features=10, out_features=1)

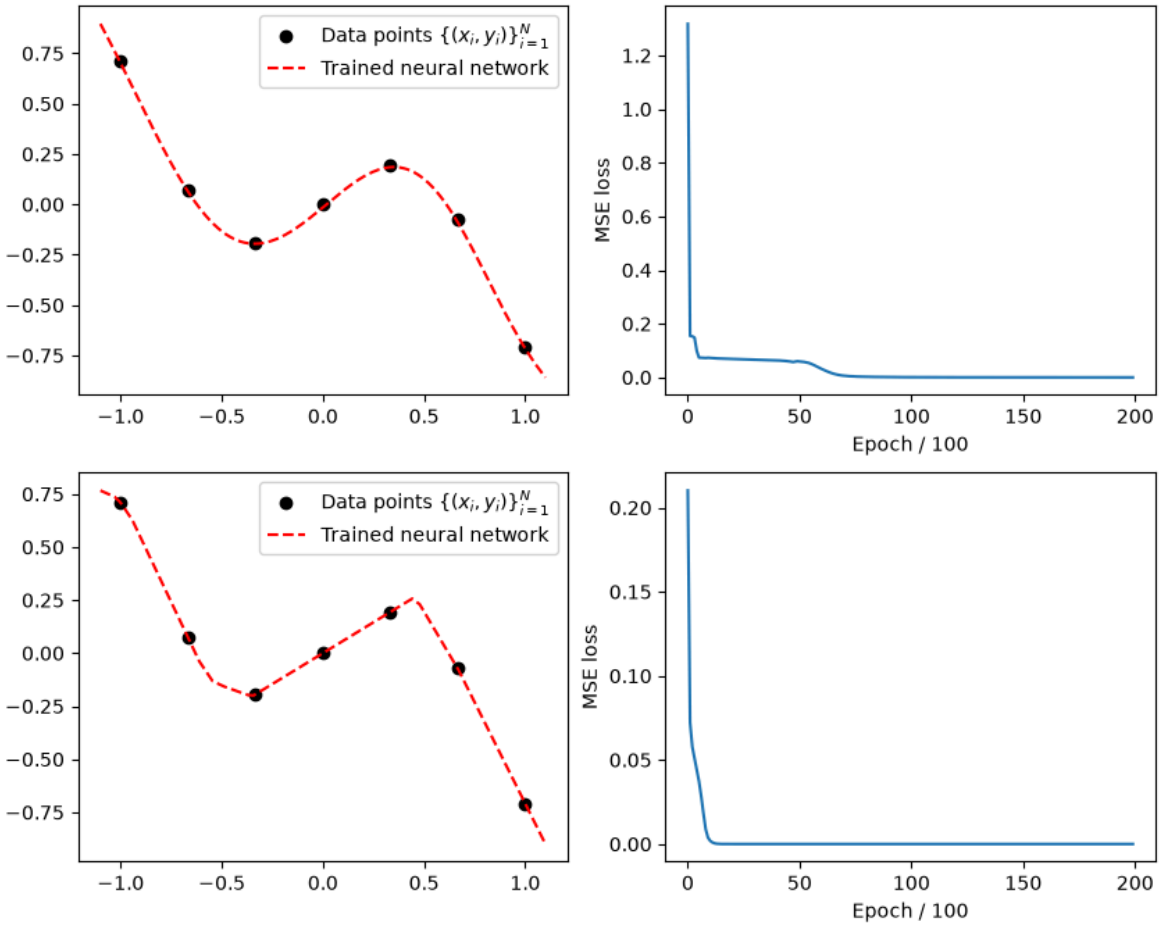
    def forward(self, x):
        x = self.linear1(x)
        x = F.relu(x)
        x = self.linear2(x)
        x = F.relu(x)
        x = self.linear3(x)
        x = F.relu(x)
        x = self.linear4(x)
        return x

sigmoid_model = SigmoidModel()
relu_model = ReluModel()

sigmoid_loss_res = train(sigmoid_model, iters=20000, log=False)
relu_loss_res = train(relu_model, iters=20000, log=False)
fig, axs = plt.subplots(2, 2, figsize=(10, 8))
x = np.linspace(-1.1, 1.1, num = 1000)
torch_x = torch.tensor(x.reshape((1000,1)), dtype=torch.float32)
y_pred_sigmoid = sigmoid_model(torch_x).detach().numpy()
y_pred_relu = relu_model(torch_x).detach().numpy()

axs[0,0].plot(xs,ys,'ko')
axs[1,0].plot(xs,ys,'ko')
axs[0,0].plot(x, y_pred_sigmoid,'r--')
axs[1,0].plot(x, y_pred_relu,'r--')
axs[0,0].legend([r'Data points $\{(x_{i},y_{i})\}_{i=1}^{N}$', r'Trained
axs[1,0].legend([r'Data points $\{(x_{i},y_{i})\}_{i=1}^{N}$', r'Trained
axs[0,1].plot(sigmoid_loss_res)
axs[1,1].plot(relu_loss_res)
axs[0,1].set_xlabel("Epoch / 100")
axs[0,1].set_ylabel("MSE loss")
axs[1,1].set_xlabel("Epoch / 100")
axs[1,1].set_ylabel("MSE loss")

plt.show()
```



c) Third, we consider different initial points (θ_0) used in training.

For an overparameterized network with ReLU activation functions at each layer and fixed widths and depth, train it with different initial point (note that the constructor of `nn.Module` will initialize the parameters randomly).

Does the initial point affect the final trained network? Give a qualitative and quantitative analysis.

Solution:

The plots show that using a different starting point the training algorithm will always converge but the speed may vary.

In [42]:
```
all_losses = []
models = []

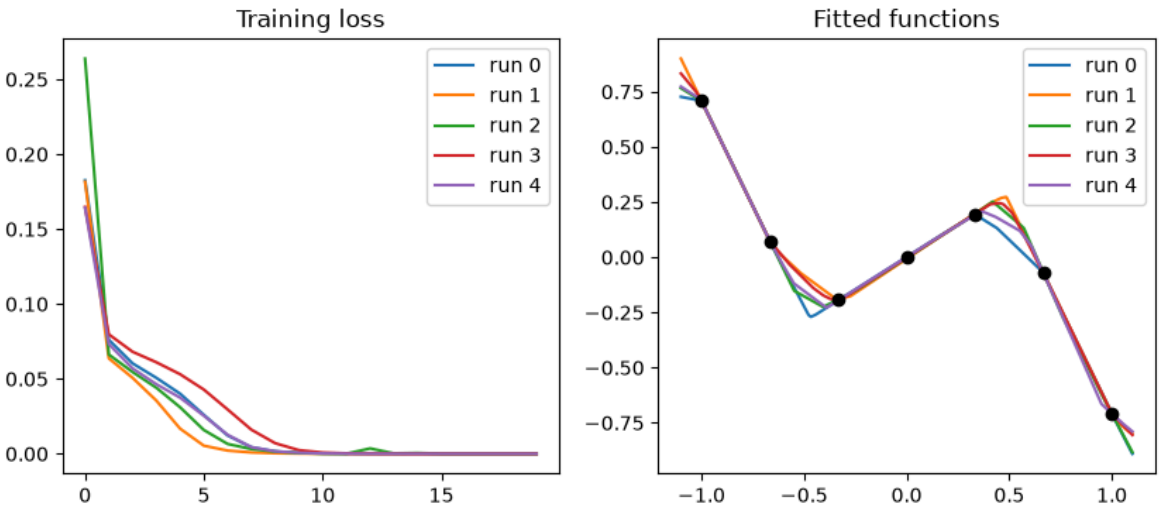
for i in range(5):
    overparameterized_model = OverparameterizedModel()
    res = train(overparameterized_model, log=False)
    all_losses.append(res)
    models.append(overparameterized_model)

fig, axs = plt.subplots(1, 2, figsize=(10, 4))
x_plot = torch.linspace(-1.1, 1.1, 1000).reshape(-1, 1)

for i, (res, m) in enumerate(zip(all_losses, models)):
    axs[0].plot(res, label=f"run {i}")
    with torch.no_grad():
```

```
        y_pred = m(x_plot)
    axs[1].plot(x_plot, y_pred, label=f"run {i}")

axs[0].set_title("Training loss")
axs[0].legend()
axs[1].set_title("Fitted functions")
axs[1].plot(xs,ys,'ko')
axs[1].legend()
plt.show()
```



d) Fourth, we consider different training algorithms.

Compare

- SGD, and
- Adam (`optimizer = optim.Adam(model.parameters(), lr=0.1)`)

for an overparameterized network with ReLU activation functions at each layer and fixed widths and depth.

You are free to experiment with the learning rates and the number of epoch.

How does these two cases differ? Give a qualitative and quantitative analysis.

Solution:

The plots show that using the Adam optimizer improved the training, giving a faster convergence.

In [43]:
```
def train_optimizer_diff(model, optimizer, log=True):
    loss_fn = nn.MSELoss()
    loss_res = []
    for epoch in range(2000):
        losses_in_epoch = []
        for batch in data_loader:

            # 1. These are the sampled batches of inputs and ground-truth
            batch_x, batch_y = batch

            # 2. Compute the model's prediction on the batch
            pred = model(batch_x)
```

```
            # 3. Compute the loss of the prediction w.r.t. ground-truth
            loss = loss_fn(pred, batch_y)

            # Save losses in a list for averaging later (not sctrictly ne
            losses_in_epoch.append(loss.item())

            # 4. Backpropagation
            loss.backward()

            # 5. One step of SGD
            optimizer.step()

            # Zero the gradients computed in the backpropagation, for sta
            optimizer.zero_grad()

        # Every 100th epoch, display progress
        if epoch % 100 == 0:
            loss_res.append(sum(losses_in_epoch)/len(losses_in_epoch))
            print('Epoch: {}\tLoss: {}'.format(epoch, sum(losses_in_epoch
    return loss_res

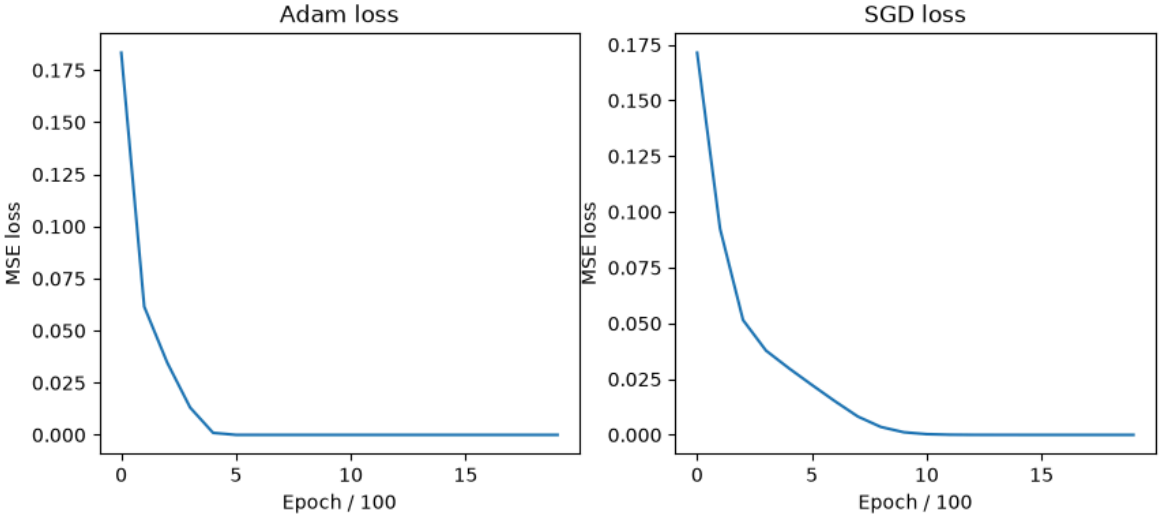
adam_model = OverparameterizedModel()
sgd_model = OverparameterizedModel()

adam_loss_res = train_optimizer_diff(adam_model, optim.Adam(adam_model.pa
sgd_loss_res = train_optimizer_diff(sgd_model, optim.SGD(sgd_model.parame

fig, axs = plt.subplots(1, 2, figsize=(10, 4))
axs[0].plot(adam_loss_res)
axs[0].set_title("Adam loss")
axs[1].plot(sgd_loss_res)
axs[1].set_title("SGD loss")
for ax in axs:
    ax.set_xlabel("Epoch / 100")
    ax.set_ylabel("MSE loss")

plt.show()
```

```
Epoch: 0        Loss: 0.1833687126636505
Epoch: 100      Loss: 0.06154957413673401
Epoch: 200      Loss: 0.034716881811618805
Epoch: 300      Loss: 0.013106283731758595
Epoch: 400      Loss: 0.0009752503829076886
Epoch: 500      Loss: 2.7969326765742153e-06
Epoch: 600      Loss: 4.7508709144494787e-07
Epoch: 700      Loss: 4.4520979258777515e-07
Epoch: 800      Loss: 4.1617627744017227e-07
Epoch: 900      Loss: 3.8715648997822427e-07
Epoch: 1000     Loss: 3.58542536105233e-07
Epoch: 1100     Loss: 3.304995175312797e-07
Epoch: 1200     Loss: 3.0336599365909933e-07
Epoch: 1300     Loss: 2.773433323000063e-07
Epoch: 1400     Loss: 2.5240680656679615e-07
Epoch: 1500     Loss: 2.2883048700350628e-07
Epoch: 1600     Loss: 2.0660107225012325e-07
Epoch: 1700     Loss: 1.8578894867005147e-07
Epoch: 1800     Loss: 1.663838986587507e-07
Epoch: 1900     Loss: 1.4835573836080584e-07
Epoch: 0        Loss: 0.1714046448469162
Epoch: 100      Loss: 0.09241980314254761
Epoch: 200      Loss: 0.05149630457162857
Epoch: 300      Loss: 0.037797413766384125
Epoch: 400      Loss: 0.029862599447369576
Epoch: 500      Loss: 0.02239134907722473
Epoch: 600      Loss: 0.015112501569092274
Epoch: 700      Loss: 0.008227712474763393
Epoch: 800      Loss: 0.0035576093941926956
Epoch: 900      Loss: 0.0011790621792897582
Epoch: 1000     Loss: 0.00032429370912723243
Epoch: 1100     Loss: 8.108889596769586e-05
Epoch: 1200     Loss: 1.915906796057243e-05
Epoch: 1300     Loss: 4.491411345952656e-06
Epoch: 1400     Loss: 1.056602741300594e-06
Epoch: 1500     Loss: 2.4844896984177467e-07
Epoch: 1600     Loss: 5.8422223503384885e-08
Epoch: 1700     Loss: 1.3757715855433617e-08
Epoch: 1800     Loss: 3.243199264701957e-09
Epoch: 1900     Loss: 7.675899360570781e-10
```



e) Fifth, we study the loss landscape of a trained network along different directions in parameter space.

Train an overparameterized network with ReLU activation functions at each layer. Denote it by $m(\cdot, \theta^*)$ where θ^* are the learned parameters. Sample Δ, with the same size as θ^*, from a standard normal distribution (and keep it fixed). Plot

$$\frac{1}{N}\sum_{i=1}^{N} L\left(m\left(x_i; \theta^* + t\Delta\right), y_i\right)$$

as a function of $t \in [-1, 1]$ (or even a larger interval).

Do the same for another network with the same widths and depth, but using 4 * sigmoid activation functions at each layer.

How does these two cases differ? Give a qualitative analysis.

Hints: You can get the parameters as a dictionary of a model by using `model.state_dict()`, updating the parameters in the dictionary, and then loading the updated parameters into the model using `model.load_state_dict(...)`. If you use this approach it is helpful to know the difference between a shallow copy and a deep copy of a dictionary You might need to cut off values of the loss that are too high to get a nice plot.

Optional: Plot in two direction at the same time, i.e. plot

$$\frac{1}{N}\sum_{i=1}^{N} L\left(m\left(x_i; \theta^* + t\Delta_t + s\Delta_s\right), y_i\right)$$

as a function of $t, s \in [-1, 1]$.

Solution:

We see from the loss landscape that we get a direction that is flat where we can find an infinite amount of minimum points due to the overparameterized character of the model.

In [44]:
```
def plot_loss_landscape(models, loss_fn, dataloader, num_points=100, alph
    fig = plt.figure(figsize=(16, 8))

    for m, (model_name, model) in enumerate(models.items()):
        # Store original parameters
        original_params = [p.clone() for p in model.parameters()]

        # Calculate two random directions
        direction1 = [torch.randn_like(p) for p in model.parameters()]
        direction2 = [torch.randn_like(p) for p in model.parameters()]

        # Normalize directions
        norm1 = torch.sqrt(sum(torch.sum(d**2) for d in direction1))
        norm2 = torch.sqrt(sum(torch.sum(d**2) for d in direction2))
        direction1 = [d / norm1 for d in direction1]
        direction2 = [d / norm2 for d in direction2]

        # Create grid
        x = np.linspace(-alpha, alpha, num_points)
```

```
        y = np.linspace(-alpha, alpha, num_points)
        X, Y = np.meshgrid(x, y)

        # Calculate loss for each point
        Z = np.zeros_like(X)
        for i in range(num_points):
            for j in range(num_points):
                # Update model parameters
                for p, d1, d2 in zip(model.parameters(), direction1, dire
                    p.data = p.data + X[i,j] * d1 + Y[i,j] * d2

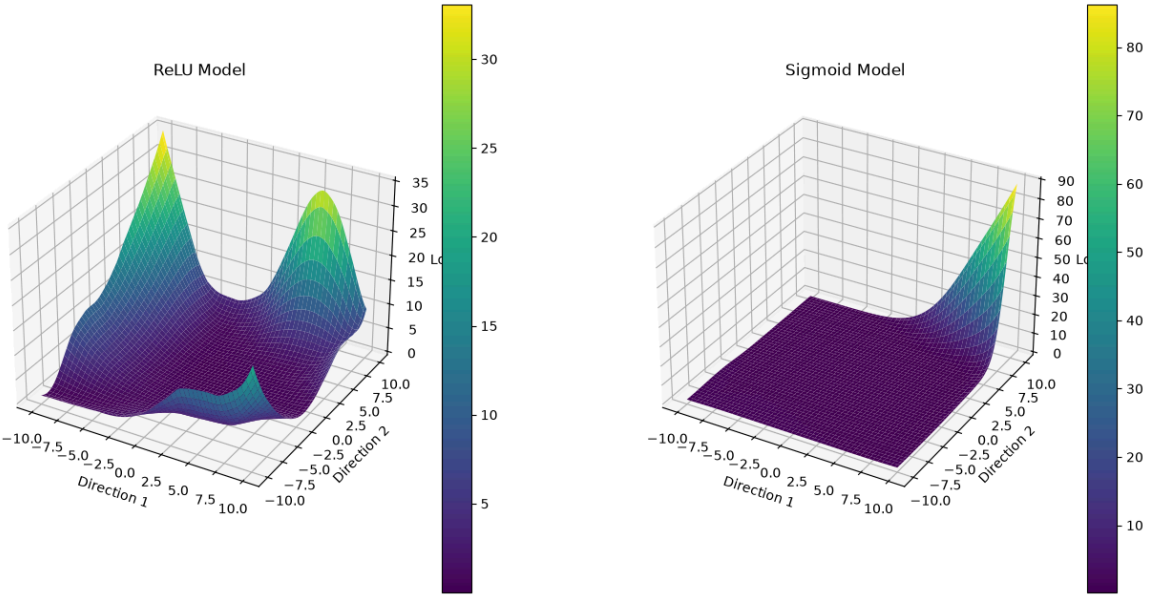
                # Calculate loss
                total_loss = 0
                num_batches = 0
                for batch in dataloader:
                    inputs, targets = batch
                    outputs = model(inputs)
                    loss = loss_fn(outputs, targets)
                    total_loss += loss.item()
                    num_batches += 1
                Z[i,j] = total_loss / num_batches

                # Reset model parameters
                for p, orig_p in zip(model.parameters(), original_params)
                    p.data = orig_p.clone()

        # Plot the loss landscape
        ax = fig.add_subplot(1, len(models), m + 1, projection='3d')
        surf = ax.plot_surface(X, Y, Z, cmap='viridis')
        ax.set_xlabel('Direction 1')
        ax.set_ylabel('Direction 2')
        ax.set_zlabel('Loss')
        ax.set_title(model_name)
        fig.colorbar(surf)

relu_model = ReluModel()
sigmoid_model = SigmoidModel()
_ = train(relu_model, log=False)
_ = train(sigmoid_model, log=False)


plot_loss_landscape({"ReLU Model": relu_model, "Sigmoid Model": sigmoid_m
```



Exercise 4: Implicit regularization via small norm in polynomial fitting in $\mathbb{R}$

Remark: You should solve this exercise using `numpy` primarily.

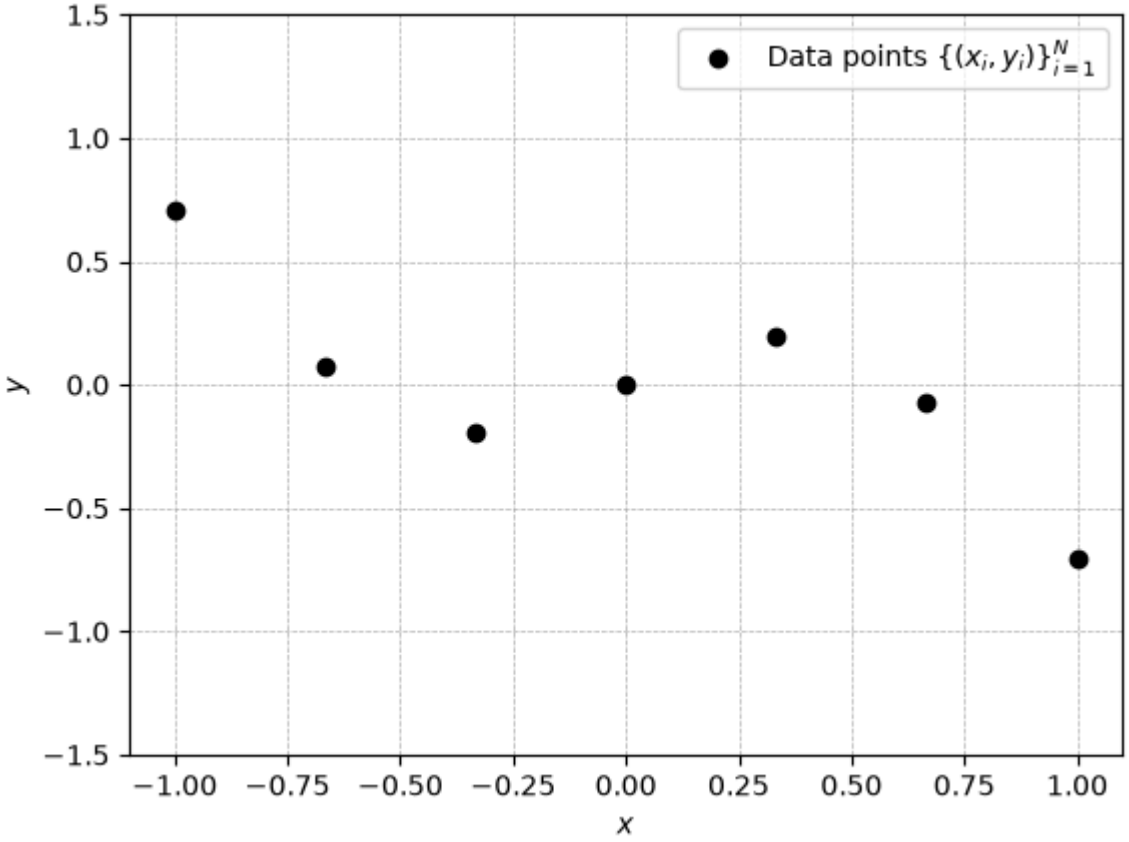
The purpose of this exercise is to study optimization problems in the overparameterized regime, i.e., when there are multiple solutions to choose from, and see how the norm of a given solution may affect generalization.

Again, we consider the data points $\{(x_i, y_i)\}_{i=1}^N \subseteq \mathbb{R} \times \mathbb{R}$, where $N = 7$, below:

In [45]:
```
N = 7 # the number of data points
xs = np.linspace(-1, 1, num=N) # the x_i's
ys = np.sin(5*xs)*np.exp(np.abs(2*xs)) / 10 # the y_i's

fig, axes = plt.subplots(dpi=100)
axes.plot(xs,ys,'ko')
axes.legend([r'Data points $\{(x_{i},y_{i})\}_{i=1}^{N}$'])
axes.set_xlabel('$x$')
axes.set_ylabel('$y$')
axes.grid(linestyle='dashed', linewidth=0.5)
axes.set_ylim(-1.5, 1.5)
```

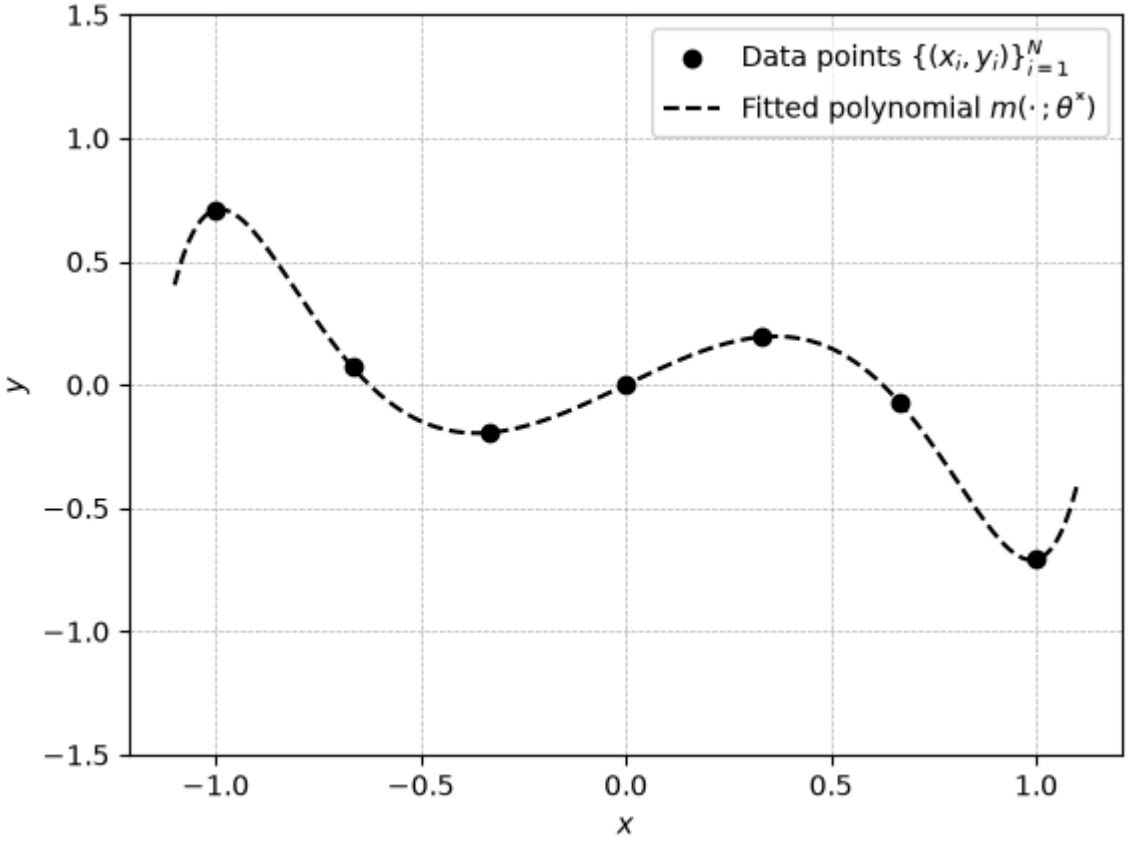
Out[45]:
```
(-1.5, 1.5)
```



We wish to fit a polynomial model $m(\cdot;\theta)$ of degree $n = 9$ (overparameterized) to the data points $\{(x_i, y_i)\}_{i=1}^N$. One solution $m(\cdot;\theta^*)$ is given below:

In [46]:
```
theta_star = np.array([0,  0.8, 0, -1.9, 0, -0.583957507, 0, 0.28, 0, 0.6
x = np.linspace(-1.1, 1.1, num = 1000)

fig, axes = plt.subplots(dpi=100)
axes.plot(xs,ys,'ko')
axes.plot(x, np.polyval(np.flip(theta_star), x),'k--')
axes.legend([r'Data points $\{(x_{i},y_{i})\}_{i=1}^{N}$', r'Fitted polyn
axes.set_xlabel('$x$')
axes.set_ylabel('$y$')
axes.grid(linestyle='dashed', linewidth=0.5)
axes.set_ylim([-1.5, 1.5]);
```



---

a) Recall from **Exercise 1 g)** that given a candidate point $a \in \mathbb{R}^{n+1}$, we can obtain a solution via the Euclidean projection formula $\theta^* = a - X^T(XX^T)^{-1}(Xa - Y)$. Initialize a randomly and compute the projection θ^* and plot the fitted polynomial $m(\cdot; \theta^*)$ as above. Vary the magnitude or norm of θ^* indirectly by multiplying a by 0, 10, 100 and 1000. Visually, how does the generalization of $m(\cdot; \theta^*)$ correspond to the norm of θ^*?

Solution:

We see that the model manages to fit the data points but the values inbetween get an increased amplitude, corresponding to the magnitude. E.g giving a far worse generalization.

In [47]:
```
n = 9

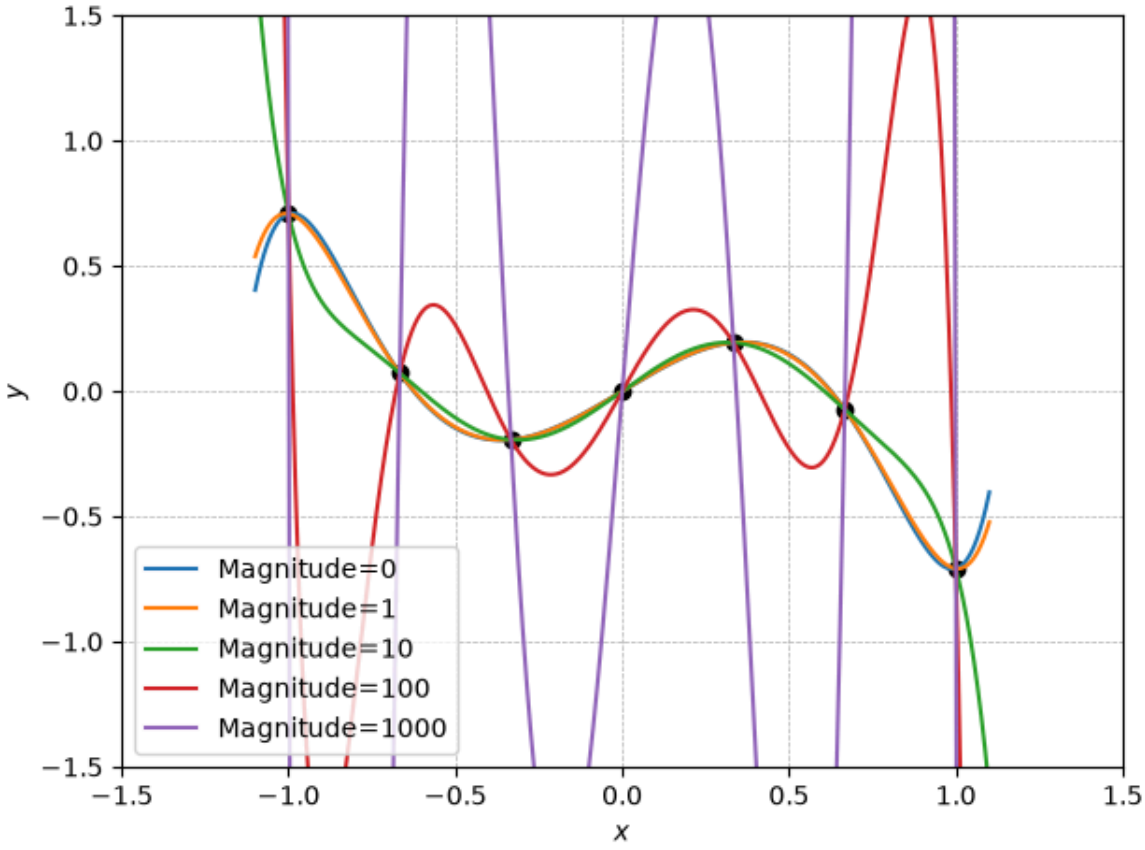
fig, axes = plt.subplots(dpi=100)
axes.plot(xs,ys,'ko')

X = np.vander(xs, n+1, increasing=True)
Y = ys

magnitudes = [0, 1, 10, 100, 1000]
direction = np.random.randn(n + 1)
for magnitude in magnitudes:
  a = direction * magnitude

  theta_star = a - X.T @ np.linalg.inv(X @ X.T) @ (X @ a - Y)
  axes.plot(x, np.polyval(np.flip(theta_star), x), label=f"Magnitude={mag
```

```
  axes.set_xlabel('$x$')
  axes.set_ylabel('$y$')
  axes.grid(linestyle='dashed', linewidth=0.5)
  axes.set_ylim(-1.5, 1.5)
  axes.set_xlim(-1.5, 1.5)
  axes.legend()
plt.tight_layout()
plt.show()
```



b) Implement the GD method for optimization problem (2) in **Exercise 1** above, starting at $\theta_0 = 0$, using step size $\gamma_k = 1/\beta$, and verify that the resulting sequence $\{\theta_k\}_{k\geq 0}$ converges to θ^*, where θ^* is the Euclidean projection starting from the candidate point $a = 0$. I.e., the GD method converges to the solution with the smallest norm. Do this by plotting $||\theta_k - \theta^*||_2^2$ as a function of k in log-linear scale.

Remark: We consider θ_k to have converged to θ^* if $||\theta_k - \theta^*||_2^2 \leq 10^{-16}$.

Solution:

In [48]:
```
def f(theta, X, Y):
  return np.linalg.norm(X @ theta - Y)**2 / len(Y)

def grad_f(theta, X, Y):
  return 2 / len(Y) * X.T @ (X @ theta - Y)

def gradient_descent(theta, n, iters =10**5):
  X = np.vander(xs, n+1, increasing=True)
  Y = ys
  beta = 2 / len(Y) * np.linalg.eigvalsh(X.T @ X).max()
```

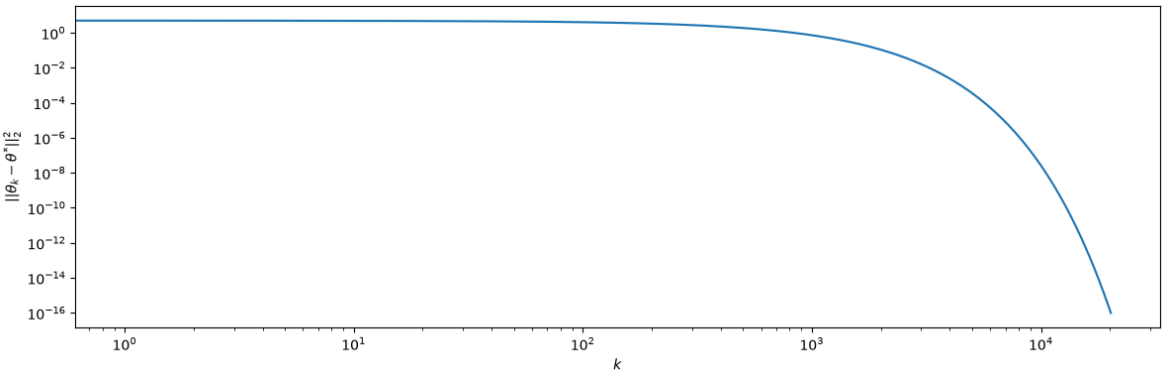
```
  gamma = 1 / beta
  a = np.zeros(n+1)
  theta_star = a - X.T @ np.linalg.inv(X @ X.T) @ (X @ a - Y)
  theta_k = theta
  res = []

  for i in range(iters):
    theta_next = theta_k - gamma * grad_f(theta_k, X, Y)
    theta_k = theta_next
    diff = np.linalg.norm(theta_k - theta_star)**2
    res.append(diff)
    if diff <= 10**-16:
          break

  return res

fig, ax = plt.subplots(figsize=(12, 4))


res_list= gradient_descent(np.zeros(n + 1), 9)
ax.loglog(res_list)
ax.set_xlabel(r"$k$")
ax.set_ylabel(r"$||\theta_{k}-\theta^{*}||_{2}^{2}$")
plt.tight_layout()
plt.show()
```



---

c) Implement the SGD method for optimization problem (1) in **Exercise 1** above, starting at $\theta_0 = 0$, using step size $\gamma_k = 0.1/\beta$, and verify that the resulting sequence $\{\theta_k\}_{k\geq 0}$ converges to θ^*, where θ^* is the Euclidean projection starting from the candidate point $a = 0$. I.e., the SGD method converges to the solution with the smallest norm. Do this by plotting $||\theta_k - \theta^*||_2^2$ as a function of k in log-linear scale.

Solution:

In [49]:
```
def f(theta, X, Y):
  return np.linalg.norm(X @ theta - Y)**2 / len(Y)

def grad_f(theta, X, Y):
  return 2 / len(Y) * X.T @ (X @ theta - Y)

def partial_grad_f(theta, X, Y, i):
  return 2 * (X[i] @ theta - Y[i]) * X[i]
```

```
def stochastic_gradient_descent(n, iters =10**2*N):
  X = np.vander(xs, n+1, increasing=True)
  Y = ys
  beta = 2 / len(Y) * np.linalg.eigvalsh(X.T @ X).max()

  gamma = 0.1 / beta

  a = np.zeros(n + 1)
  theta_star = a - X.T @ np.linalg.inv(X @ X.T) @ (X @ a - Y)

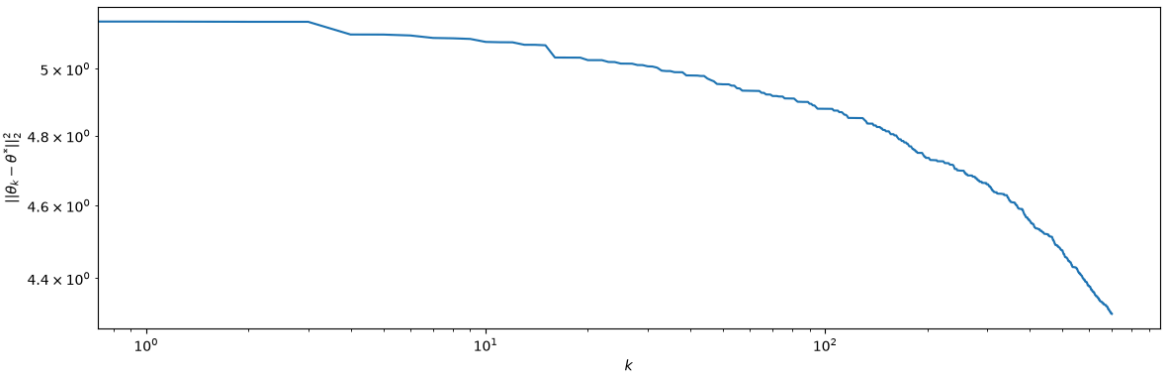
  theta_k = np.zeros(n+1)
  res = []

  for i in range(iters):
    j = np.random.randint(0, len(Y))
    theta_next = theta_k - gamma * partial_grad_f(theta_k, X, Y, j)
    theta_k = theta_next
    diff = np.linalg.norm(theta_k-theta_star)**2
    res.append(diff)
    if diff <= 10**-16:
      break


  return res

fig, ax = plt.subplots(figsize=(12, 4))


res_list= stochastic_gradient_descent(9)
ax.loglog(res_list)
ax.set_xlabel(r"$k$")
ax.set_ylabel(r"$||\theta_{k}-\theta^{*}||_{2}^{2}$")
plt.tight_layout()
plt.show()
```



---

d) The ADAM optimization method is given by:

- Pick $\theta_0 \in \mathbb{R}^{n+1}$, $\hat{m}_0 = \hat{v}_0 = 0 \in \mathbb{R}^{n+1}$, $b_m, b_v \in (0,1)$ and $\gamma, \epsilon > 0$.
- For $k \geq 0$:
  - Pick $i \in \{1, \ldots, N\}$ uniformly at random
  - Set $g_{k+1} = \nabla f_i(\theta_k)$
  - Set $\hat{m}_{k+1} = b_m \hat{m}_k + (1 - b_m) g_{k+1}$
  - Set $\hat{v}_{k+1} = b_v \hat{v}_k + (1 - b_v) g_{k+1}^2$ (element-wise square)

- Set $m_{k+1} = \hat{m}_{k+1}/(1 - b_m^{k+1})$
- Set $v_{k+1} = \hat{v}_{k+1}/(1 - b_v^{k+1})$
- Set $\theta_{k+1} = \theta_k - \gamma m_{k+1}./(\sqrt{v_{k+1}} + \epsilon 1)$ (element-wise division and square root)

Use the suggested values:

- $b_m = 0.9$
- $b_v = 0.999$
- $\gamma = 0.001$
- $\epsilon = 10^{-8}$

Implement the ADAM optimization method for optimization problem (1) in **Exercise 1** above, starting at $\theta_0 = 0$. Does ADAM converge to the minimum norm solution? Plot the resulting model.

Solution:

The plot shows that ADAM does not converge to the minimum norm solution.

In [50]:
```
def f(theta, X, Y):
  return np.linalg.norm(X @ theta - Y)**2 / len(Y)

def grad_f(theta, X, Y):
  return 2 / len(Y) * X.T @ (X @ theta - Y)

def partial_grad_f(theta, X, Y, i):
  return 2 * (X[i] @ theta - Y[i]) * X[i]

def ADAM_optimization(n, b_m, b_v, gamma, eps, iters=6*10**4):
  X = np.vander(xs, n+1, increasing=True)
  Y = ys

  a = np.zeros(n + 1)
  theta_star = a - X.T @ np.linalg.inv(X @ X.T) @ (X @ a - Y)

  theta_k = np.zeros(n+1)
  m_hat_k = np.zeros(n+1)
  v_hat_k = np.zeros(n+1)
  res = []

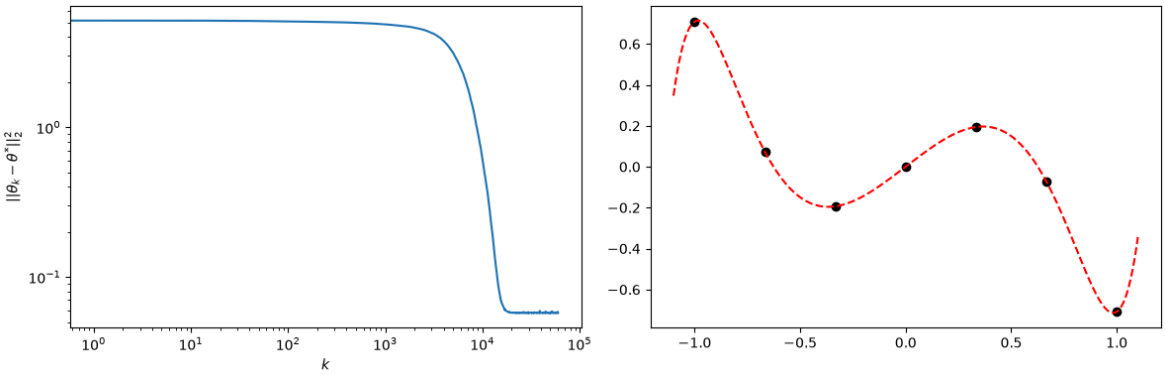
  for i in range(iters):
    j = np.random.randint(0, len(Y))
    g_next = partial_grad_f(theta_k, X, Y, j)
    m_hat_k = b_m * m_hat_k + (1-b_m) * g_next
    v_hat_k = b_v * v_hat_k +(1-b_v) * np.square(g_next)
    m_next = m_hat_k  / (1-b_m**(i+1))
    v_next = v_hat_k / (1-b_v**(i+1))
    theta_k = theta_k - gamma * np.divide(m_next, (np.sqrt(v_next) + eps)
    diff = np.linalg.norm(theta_k-theta_star)**2
    res.append(diff)
    if diff <= 10**-16:
      break

  return res, (X, Y, theta_k)
```

```
fig, axs = plt.subplots(1, 2, figsize=(12, 4))

res_list, (X, Y, theta_k)= ADAM_optimization(9, 0.9, 0.999, 0.001, 10**-8
fitted_model = f(X, Y, theta_k)
axs[0].loglog(res_list)
axs[0].set_xlabel(r"$k$")
axs[0].set_ylabel(r"$||\theta_{k}-\theta^{*}||_{2}^{2}$")
axs[1].plot(xs,ys,'ko')
axs[1].plot(x, np.polyval(np.flip(theta_k), x),'r--')

plt.tight_layout()
plt.show()
```



---

e) The ADAM optimization method scales the gradient independently in each coordinate. We call such methods diagonally scaled methods.

Now, recall the scaled projection problem from **Exercise 1 g)** above: given a candidate point $a \in \mathbb{R}^{n+1}$, we can obtain a solution via the scaled projection formula $\theta^* = a - H^{-1}X^T(XH^{-1}X^T)^{-1}(Xa - Y)$.

Consider the candidate point $a \in \mathbb{R}^{n+1}$ and the highly skewed diagonal scaling matrix $H \in \mathbb{S}_{++}^{n+1}$ below:

In [51]:
```
a = np.zeros(n + 1)
H_inv = np.diag(np.logspace(-10, 10, num = n + 1)) # the inverse of H
```

Plot the data points $\{(x_i, y_i)\}_{i=1}^N$ and the fitted polynomial $m(\cdot; \theta^*)$ in the same figure using the scaled projection formula. Visually, does the fitted polynomial $m(\cdot; \theta^*)$ seem to generalize well?

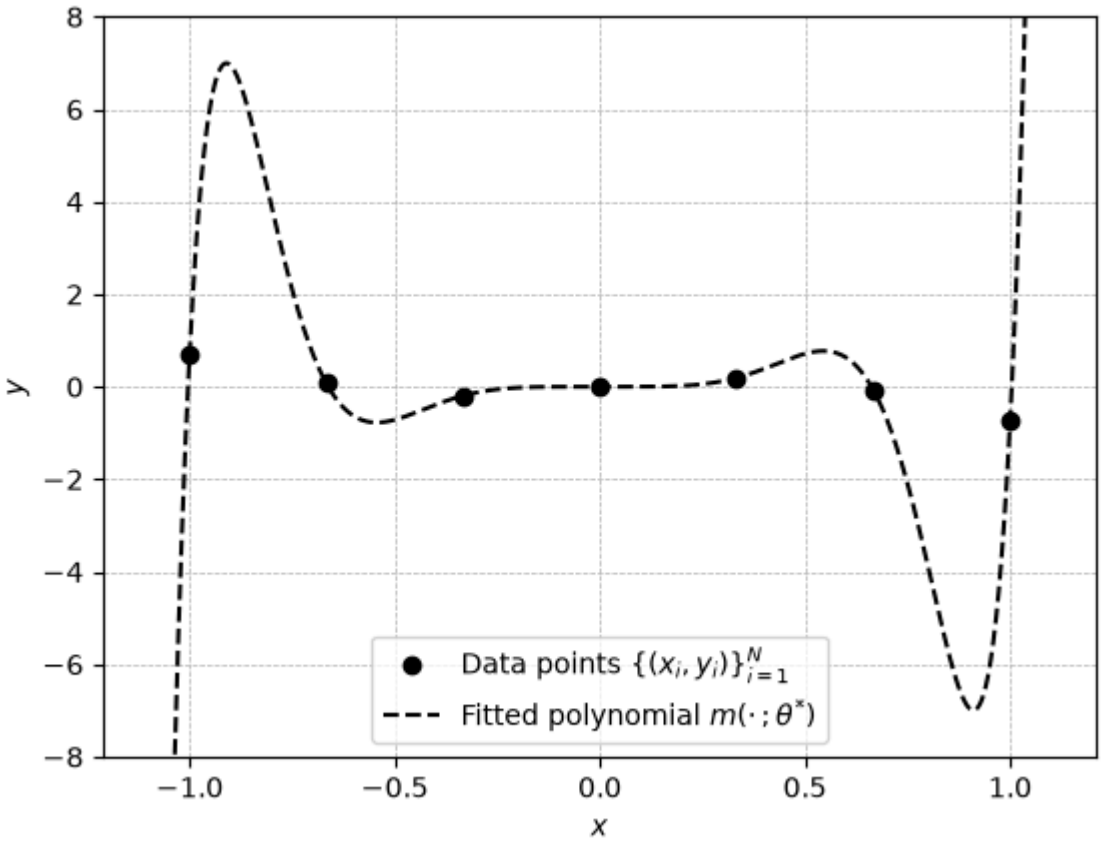
Solution:

No, it does not generalize well as it appears to go to negative infinity when x is less than our definition set and to positive infinity when x is greater than our definition set.

In [52]:
```
theta_star = a - H_inv @ X.T @ np.linalg.inv(X @ H_inv @ X.T) @ (X @ a -
x = np.linspace(-1.1, 1.1, num = 1000)

fig, axes = plt.subplots(dpi=100)
axes.plot(xs,ys,'ko')
axes.plot(x, np.polyval(np.flip(theta_star), x),'k--')
axes.legend([r'Data points $\{(x_{i},y_{i})\}_{i=1}^{N}$', r'Fitted polyn
```

```
axes.set_xlabel('$x$')
axes.set_ylabel('$y$')
axes.grid(linestyle='dashed', linewidth=0.5)
axes.set_ylim(-8, 8)
plt.show()
```



---

f) The scaled GD method is given by

- Pick $\theta_0 \in \mathbb{R}^{n+1}$ and $H \in \mathbb{S}_+^{n+1}$
- For $k \geq 0$:
  - Set $\theta_{k+1} = \theta_k - \gamma_k H^{-1} \nabla f(\theta_k)$ for some $\gamma_k > 0$.

For numerical reasons (i.e., so that the algorithm converges in reasonable time), consider the less skewed diagonal scaling matrix below:

In [53]:
```
H_inv = np.diag(np.logspace(-3, 0, num = n + 1)) # the inverse of H
```

Starting at $\theta_0 = 0$ and using the step size $\gamma_k = 1/\beta_H$, show that the scaled GD method converges to the scaled projection point
$\theta^* = a - H^{-1}X^T(XH^{-1}X^T)^{-1}(Xa - Y)$ for $a = 0$.

Here, β_H is given by

$$\beta_H = \frac{2}{N}\left\|H^{-\frac{1}{2}}X^T X H^{-\frac{1}{2}}\right\|_2, \tag{20}$$

where $H^{-\frac{1}{2}}$ is the symmetric square root of H^{-1}.

Solution:

Fill in your solution here!

In [54]:
```
def scaled_gradient_descent(theta_k, n, iters=10**6):
  X = np.vander(xs, n+1, increasing=True)
  Y = ys
  beta_H = (2 / len(Y) ) * np.linalg.norm(np.square(H_inv) @ X.T @ X @ np

  theta_star = a - H_inv @ X.T @ np.linalg.inv(X @ H_inv @ X.T) @ (X @ a

  gamma = 1 / beta_H

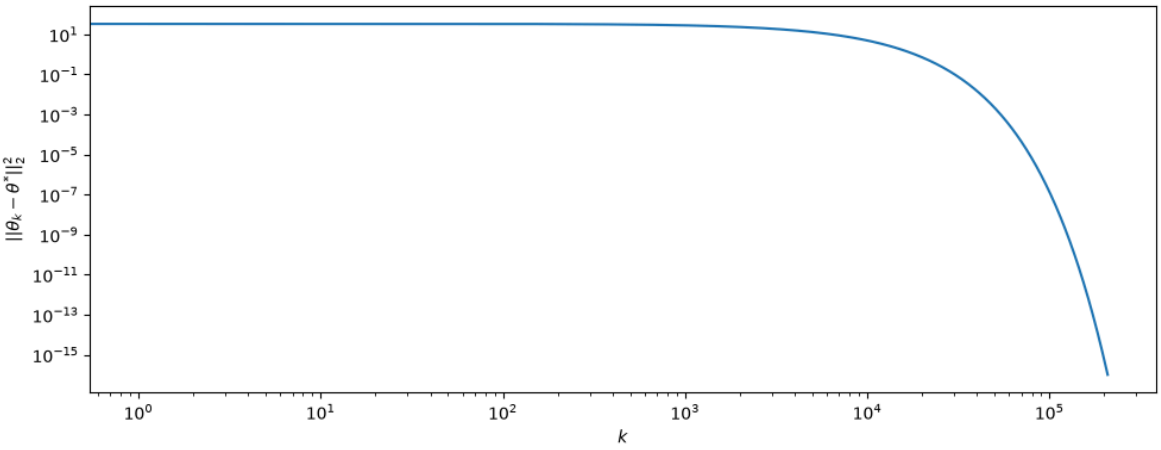
  res = []

  for i in range(iters):
    theta_k = theta_k - gamma * H_inv @ grad_f(theta_k, X, Y)
    diff = np.linalg.norm(theta_k - theta_star)**2
    res.append(diff)
    if diff <= 10**-16:
      break

  return res

fig, ax = plt.subplots(figsize=(10, 4))

res_list= scaled_gradient_descent(np.zeros(n+1), 9)
ax.loglog(res_list)
ax.set_xlabel(r"$k$")
ax.set_ylabel(r"$||\theta_{k}-\theta^{*}||_{2}^{2}$")
plt.tight_layout()
plt.show()
```



Exercise 5: Implicit regularization via flatness

Remark: You should solve this exercise using `numpy` primarily.

It has been observed in the literature that flat mimima tend to lead to better generalization than sharp minima. In this exercise, we empirically verfiy that SGD tends to favour flat mimima.

Consider the finite-sum problem problem

$$\underset{x\in\mathbb{R}}{\text{minimize}} \qquad f(x) = \frac{1}{N}\sum_{i=1}^{N} f_i(x) \tag{21}$$

where the f_i's and f are given below: (the derivatives are also given for convenience.)

In [55]:
```
from functools import partial

def loss_term(phase, x):
    if x < 50/11:
        value = x**2
    elif x < 50/9:
        value = 100*(x - 5)**2
    else:
        value = x**2
    return value + 20*np.sin(10*x + phase)*np.cos(2*(x - 5))

def derivative_of_loss_term(phase, x):
    if x < 50/11:
        value = 2*x
    elif x < 50/9:
        value = 200*(x - 5)
    else:
        value = 2*x
    return value + 20*(10*np.cos(10*x + phase)*np.cos(2*(x - 5)) - 2*np.s

N = 2000 # the number of summands
np.random.seed(0)
phases = 2 * np.pi * np.random.rand(N)

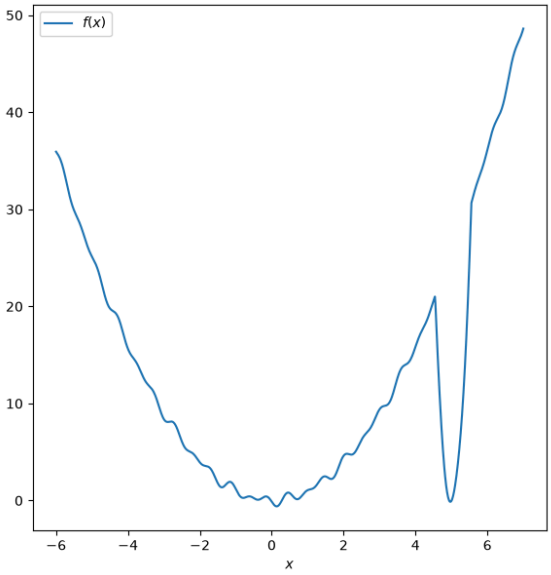
# The induvidual summands f_i as a list
f_i_s = list(map(lambda phase : np.vectorize(partial(loss_term, phase)),

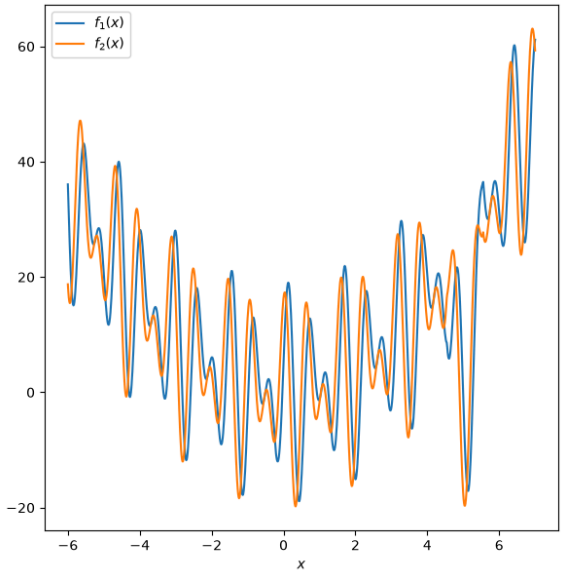
# The function f
f = np.vectorize(lambda x : np.mean([f_i(x) for f_i in f_i_s]))

# Derivatives are provided for convenience
derivative_f_i_s = list(map(lambda phase : np.vectorize(partial(derivativ
derivative_f = np.vectorize(lambda x : np.mean([d_f_i(x) for d_f_i in der

x = np.linspace(-6, 7, num=1000)
y_1 = f_i_s[0](x)
y_2 = f_i_s[1](x)
y = f(x)

fig, axes = plt.subplots(1, 2, figsize=(15,7))
axes[0].plot(x, y)
axes[0].legend(["$f(x)$"])
axes[0].set_xlabel('$x$')
axes[1].plot(x, y_1)
axes[1].plot(x, y_2)
axes[1].legend(["$f_{1}(x)$", "$f_{2}(x)$"])
axes[1].set_xlabel('$x$')
plt.show()
```





Note that the function f has a flat minimum close to 0 and a sharp minimum close to 5. Implement both GD and SGD for this problem:

- Start at $x_0 = 5$, i.e., close to the sharp minimum.
- Let both GD and SGD go for 2000 iterations.
- Use the same constant step size $\gamma_k = 0.001$ for SGD and GD.
- You should notice that SDG manages to escape the sharp minimum and find the flat minimum (most of the time), while GD does not. Illustrate this by plotting x_k as a function of k for both methods. (Common mistake: We are not asking for $f(x_k)$ vs x_k.)

Give some brief comments.

Solution:

The plot clearly shows that SGD favors the flat minimum, in comparison to GD.

In [56]:
```
def gradient_descent(iters=2000):

  gamma = 0.001
  x_k = 5
  res = [x_k]

  for i in range(iters):
    x_k = x_k - gamma * derivative_f(x_k)
    res.append(x_k)
  return res

def stochastic_gradient_descent(iters=2000):
  gamma = 0.001
  x_k = 5
  res = [x_k]

  for i in range(iters):
    j = np.random.randint(0, len(derivative_f_i_s))
    x_k = x_k - gamma * derivative_f_i_s[j](x_k)
    res.append(x_k)
```

```
  return res

gd_res = gradient_descent()
sgd_res = stochastic_gradient_descent()

fig, axes = plt.subplots(1, 1, figsize=(12,4))
axes.plot(gd_res, label="GD")
axes.plot(sgd_res, label="SGD")
axes.legend()
axes.set_xlabel(r'$k$')
```

Out[56]: Text(0.5, 0, 'k')

